

Production C++ Services and Cloud-Native Infrastructure

Containers, Kubernetes, Observability, CI/CD, Cloud Deployment, and Site Reliability Engineering

Dr. Charles Dorner

C++ From Beginner to Expert — Book VIII

First edition · 2026

About this volume

Book VIII takes distributed C++ services into production infrastructure. The emphasis is on connecting application contracts to deployment, identity, networking, observability, scaling, security, reliability, and recovery. HarborDispatch remains the recurring instructional service.

How to use this book

For every mechanism ask: what requirement does it serve, what state does it control, what evidence proves the outcome, and how is the change reversed or recovered? Use isolated labs before adapting examples to real infrastructure. Provider APIs and managed-service behavior change; verify current vendor documentation before deployment.

Chapter 1 — Production Service Engineering

This chapter develops **production service engineering** as a production mechanism whose application and platform responsibilities can be tested and recovered.

1.1 Application Boundary

1.1.1 Engineering model

Production Service Engineering — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume

unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

1.1.2 Production verification

Production Service Engineering verification — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Return distinct application outcomes for invalid and saturated requests

Keep business rejection separate from platform health so a full queue is not reported as a successful job.

main.cpp

```
#include <array>
#include <string_view>
#include <iostream>

int main() {
    enum class Outcome{accepted,invalid,busy};unsigned queued=0;
    auto submit=[&](int n){if(n<0||n>100)return Outcome::invalid;if(queued==2)return
Outcome::busy;++queued;return Outcome::accepted;};
    auto name=[](Outcome o)->std::string_view{return
o==Outcome::accepted?"accepted":o==Outcome::invalid?"invalid":"busy";};
    for(int n:std::array{2,-1,3,4})std::cout<<name(submit(n))<<'\\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The range check enforces the application input contract; the queue check enforces admission capacity. Neither outcome depends on whether the process runs inside a container.

A platform probe must not count busy as a completed business request. Explicit outcomes distinguish a nonretryable invalid input from a bounded overload response.

Expected behavior or output

```
accepted
invalid
accepted
busy
```

Concrete experiment

Release one queue slot and retry 4 with the same operation identity. Admission may succeed, but the earlier busy response must not be counted as accepted work.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

1.2 Platform Boundary

1.2.1 Engineering model

Production Service Engineering — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

1.2.2 Production verification

Production Service Engineering verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Give the platform finite resources and a non-root identity

This Pod specifies platform permissions and resource budgets around a separately built C++ application.

example.py

```
import pathlib,yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
pod=docs[0];c=pod['spec']['containers'][0]
assert pod['spec']['automountServiceAccountToken'] is False
assert c['securityContext']['readOnlyRootFilesystem'] is True
assert c['resources']['limits']['memory']=='128Mi'
print('offline contract: non-root, no API token, read-only root, 128Mi limit')
```

example.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: harbor-contract
spec:
  automountServiceAccountToken: false
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    seccompProfile: {type: RuntimeDefault}
  containers:
    - name: app
      image: harbor-service:lab
      imagePullPolicy: Never
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities: {drop: [ALL]}
      resources:
        requests: {cpu: 100m, memory: 64Mi}
        limits: {cpu: 500m, memory: 128Mi}
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

runAsUser selects the process UID. Dropping capabilities and disallowing privilege escalation reduce authority, but they do not validate application input or make the program memory-safe.

The lab image must already exist on the target node because imagePullPolicy is Never. The offline check verifies selected manifest fields, not scheduler acceptance, image availability, or runtime enforcement.

Expected behavior or output

```
offline contract: non-root, no API token, read-only root, 128Mi limit
```

Concrete experiment

Run against a read-only root and move required writable data to an explicit volume. Do not disable the restriction just because a log path was hardcoded.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-RESOURCES, K8S-SECURITY. See the added references and revision index.

1.3 Failure and Recovery

1.3.1 Engineering model

Production Service Engineering — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

1.3.2 Production verification

Production Service Engineering verification — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally

complete.

NEW SECTION EXAMPLE

Recover pending work after a process-shaped interruption

Model accepted intent separately from volatile execution state and reconstruct pending work after restart.

main.cpp

```
#include <map>
#include <optional>
#include <string>
#include <iostream>

int main() {
    struct Job{int input;std::optional<int> result;};std::map<std::string,Job>
durable{{"a",{7,{}}}};
    bool volatile_in_flight=true;volatile_in_flight=false; // process state lost
    for(auto&[id,j]:durable)if(!j.result){volatile_in_flight=true;j.result=j.input*j.input;vol
atile_in_flight=false;}
    std::cout<<"recovered="<<*durable.at("a").result<<"
in-flight="<<std::boolalpha<<volatile_in_flight<<'\\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

A missing result identifies an obligation that can be redispached in this persistence model. The volatile flag cannot be the only record of accepted work because it disappears with the process.

The map is a supplied persistence model, not a durable database. A production dispatcher needs persisted operation IDs, deduplicated effects, and actual restart tests; restarting a container cannot reconstruct lost intent.

Expected behavior or output

```
recovered=49 in-flight=false
```

Concrete experiment

Mark a result completed before the modeled interruption. Recovery should skip it rather than applying the operation again.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Apply production service engineering to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 2 — Containerizing C++ Services

This chapter develops **containerizing c++ services** as a production mechanism whose application and platform responsibilities can be tested and recovered.

2.1 Defining the Contract

2.1.1 Engineering model

Containerizing C++ Services — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop

admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

2.1.2 Production verification

Containerizing C++ Services verification — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Keep the application in the foreground as the container's main process

Compile a foreground C++ program and show the exec-form container entrypoint that starts it directly.

main.cpp

```
#include <iostream>
int main(int argc, char**argv){if(argc!=2)return 2;std::cout<<"foreground
argument="<<argv[1]<<'\\n';}
```

Dockerfile

```
FROM debian:bookworm-slim AS build
RUN apt-get update && apt-get install -y --no-install-recommends g++ && rm -rf
/var/lib/apt/lists/*
WORKDIR /src
COPY main.cpp .
RUN g++ -std=c++20 -O2 main.cpp -o app
FROM debian:bookworm-slim
COPY --from=build /src/app /usr/local/bin/app
ENTRYPOINT ["/usr/local/bin/app"]
CMD ["hello"]
```

check.sh

```
set -eu
g++ -std=c++20 main.cpp -o app
./app hello
```

Run or inspect

```
sh check.sh
docker build -t harbor-entrypoint .
docker run --rm harbor-entrypoint
```

Explanation

The program does not daemonize or fork away. Exec-form ENTRYPOINT names the executable directly, while CMD supplies replaceable default arguments; the container lifecycle can therefore follow that application process.

The native test verifies only argument handling. Image building and execution require Docker; the base tags shown are mutable tutorial tags rather than production digest locks.

Expected behavior or output

```
foreground argument=hello
```

Concrete experiment

On a Docker lab host, run `docker run --rm harbor-entrypoint goodbye`. The argument changes without editing the entrypoin.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: DOCKER-MULTISTAGE. See the added references and revision index.

2.2 Implementing the Mechanism

2.2.1 Engineering model

Containerizing C++ Services — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

2.2.2 Production verification

Containerizing C++ Services verification — Implementing the Mechanism

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume

unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Build a C++ executable in one stage and copy only the runtime artifact

Keep compiler packages out of the final image while matching runtime libraries to the build environment.

main.cpp

```
#include <iostream>
int main(){std::cout<<"container artifact ready\n";}
```

Dockerfile

```
FROM debian:bookworm-slim AS builder
RUN apt-get update && apt-get install -y --no-install-recommends g++ && rm -rf
/var/lib/apt/lists/*
WORKDIR /work
COPY main.cpp .
RUN g++ -std=c++20 -O2 main.cpp -o harbor
FROM debian:bookworm-slim AS runtime
RUN apt-get update && apt-get install -y --no-install-recommends libstdc++6 && rm -rf
/var/lib/apt/lists/*
COPY --from=builder /work/harbor /usr/local/bin/harbor
USER 10001:10001
ENTRYPOINT ["/usr/local/bin/harbor"]
```

check.py

```
from pathlib import Path
s=Path('Dockerfile').read_text()
assert 'COPY --from=builder /work/harbor' in s
assert s.count('FROM ')==2
assert 'USER 10001:10001' in s
print('offline image structure: builder, runtime library, non-root executable')
```

Run or inspect

```
python3 check.py
docker build -t harbor-runtime .
docker run --rm harbor-runtime
```

Explanation

The named builder stage contains g++ and source. COPY --from imports only the executable, not the build tree or package cache, into the runtime stage.

libstdc++6 supplies the dynamically linked C++ runtime. A scratch image would not automatically contain the required loader and libraries. The structural check is not an image build or runtime compatibility test.

Expected behavior or output

offline image structure: builder, runtime library, non-root executable

Concrete experiment

Build on a Docker host and inspect runtime dependencies. Remove `libstdc++6` only after proving that the binary no longer requires it.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: DOCKER-MULTISTAGE, LINUX-LOADER. See the added references and revision index.

2.3 Verifying Production Behavior

2.3.1 Engineering model

Containerizing C++ Services — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level

observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

2.3.2 Production verification

Containerizing C++ Services verification — Verifying Production Behavior

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Test container exit status rather than just image creation

Expect one successful invocation and one deliberate input rejection in a runtime smoke test.

main.cpp

```
#include <string_view>
#include <iostream>
int main(int
argc, char**argv){if (argc!=2 || std::string_view(argv[1])!="check"){std::cerr<<"invalid
mode\n"; return 2;}std::cout<<"healthy\n";}
```

local_test.sh

```
set -eu
g++ -std=c++20 main.cpp -o app
test "$(/app check)" = healthy
if ./app wrong >out.txt 2>err.txt; then exit 1; else status=$?; fi
test "$status" = 2
grep -qx 'invalid mode' err.txt
printf 'success output and rejection exit2 verified\n'
```

container_test.sh

```
set -eu
: "${IMAGE:?Set the built image whose entrypoint is app}"
test "$(docker run --rm "$IMAGE" check)" = healthy
if docker run --rm "$IMAGE" wrong >out.txt 2>err.txt; then exit 1; else status=$?; fi
test "$status" = 2
```

Run or inspect

```
sh local_test.sh
# With a separately built matching image:
IMAGE=harbor-check sh container_test.sh
```

Explanation

The local test checks application stdout and its failure exit code. A successful container build proves neither of these runtime behaviors.

The second script repeats the contract through Docker when the matching image exists; it is not part of the local pass record. Service qualification also needs the actual network path and readiness semantics.

Expected behavior or output

success output and rejection exit2 verified

Concrete experiment

Change the invalid-mode exit code to zero and verify that the smoke test fails. Process startup must not hide a rejected request.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: DOCKER-RUN. See the added references and revision index.

Worked example — Apply containerizing c++ services to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 3 — Engineering Production Container Images

This chapter develops **engineering production container images** as a production mechanism whose application and platform responsibilities can be tested and recovered.

3.1 Application Boundary

3.1.1 Engineering model

Engineering Production Container Images — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation

must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

3.1.2 Production verification

Engineering Production Container Images verification — Application

Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Remove an implicit shell dependency from the application contract

Read configuration natively instead of shelling out to `cat`, reducing the minimal runtime's dependencies.

main.cpp

```
#include <fstream>
#include <string>
#include <iostream>

int main() {
    {std::ofstream out("config.txt");out<<"workers=4\n";if(!out)return 1;}
    std::ifstream in("config.txt");std::string line;
    if(!std::getline(in,line)||line!="workers=4")return 2;
    std::cout<<"native configuration read: "<<line<<"\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

`std::ifstream` opens the file through the C++ runtime. There is no shell, `cat` process, or shell-quoting boundary, so the application's runtime requirements are smaller and explicit.

The fixture writes its own disposable input for deterministic output. Production configuration still needs size, syntax, permission, and reload policies; a standard stream does not validate those automatically.

Expected behavior or output

```
native configuration read: workers=4
```

Concrete experiment

Remove `config.txt` before reading and require a reported startup failure. Do not silently use a default when a mandatory production file is absent.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

3.2 Platform Boundary

3.2.1 Engineering model

Engineering Production Container Images — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

3.2.2 Production verification

Engineering Production Container Images verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Exclude build caches and secrets from the container context

Inspect `.dockerignore` before sending a build context to a daemon or remote builder.

`.dockerignore`

```
.git
build/
out/
.env
*.pem
secrets/
*.core
```

`check.py`

```
from pathlib import Path
patterns=set(Path('.dockerignore').read_text().splitlines())
required={''.git','build/',''.env','*.pem','secrets/','*.core'}
assert required<=patterns
print('context exclusions include VCS, builds, secrets, and core dumps')
```

Run or inspect

```
python3 check.py
```

Explanation

`COPY . .` can include unintended files from the build context. The patterns exclude common large and sensitive inputs before the Dockerfile copies source.

This is an offline policy check, not proof that no secret exists. Credentials can have unexpected names, and build-time secrets should use dedicated secret mounts rather than build arguments or image layers.

Expected behavior or output

```
context exclusions include VCS, builds, secrets, and core dumps
```

Concrete experiment

Create dummy nonsecret data under `secrets/example.txt` and inspect the effective context on a Docker lab host. Do not test using real credentials.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: DOCKER-CONTEXT. See the added references and revision index.

3.3 Failure and Recovery

3.3.1 Engineering model

Engineering Production Container Images — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

3.3.2 Production verification

Engineering Production Container Images verification — Failure and Recovery

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Fail a release check when a runtime dependency is unresolved

Inspect a locally built ELF executable's loader requirements before assuming that copying one file creates a runtime.

main.cpp

```
#include <iostream>
int main(){std::cout<<"runtime check\n";}
```

check.sh

```
set -eu
g++ -std=c++20 main.cpp -o app
ldd ./app >dependencies.txt
if grep -q 'not found' dependencies.txt; then cat dependencies.txt >&2; exit 1; fi
./app
readelf -l app >program-headers.txt
grep -q INTERP program-headers.txt
printf 'loader path recorded; host dependencies resolved\n'
```

Run or inspect

```
sh check.sh
```

Explanation

ldd reports resolved host libraries and the check rejects a not-found dependency. readelf records the program interpreter, another required part of a dynamically linked runtime.

Only the trusted executable built by this exercise is passed to ldd; do not use it on untrusted binaries. Host resolution does not prove the same libraries exist in the final container.

Expected behavior or output

```
runtime check
loader path recorded; host dependencies resolved
```

Concrete experiment

Move the binary into a minimal test image and verify startup there. Keep host dependency evidence separate from container runtime evidence.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: LINUX-LOADER. See the added references and revision index.

Worked example — Apply engineering production container images to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```

#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}

```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 4 — Registries, Artifacts, and Software Supply Chains

This chapter develops **registries, artifacts, and software supply chains** as a production mechanism whose application and platform responsibilities can be tested and recovered.

4.1 Defining the Contract

4.1.1 Engineering model

Registries, Artifacts, and Software Supply Chains — Defining the Contract

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation

must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

4.1.2 Production verification

Registries, Artifacts, and Software Supply Chains verification — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Require content identity at the artifact promotion boundary

Distinguish a mutable tag from a digest-qualified image reference at promotion.

example.py

```
import re
refs=['registry.example/harbor:latest','registry.example/harbor@sha256: '+'a'*64]
def immutable_name(s):return re.fullmatch(r'^\s@]+@sha256:[0-9a-f]{64}',s) is not None
assert not immutable_name(refs[0]) and immutable_name(refs[1])
print('tag rejected; digest-shaped reference accepted for further verification')
```

Run or inspect

```
python3 example.py
```

Explanation

A digest-shaped reference names content rather than a moving tag. The regular expression checks syntax only; the all-a digest is deliberately a nonresolving fixture, not a published image.

Promotion must also retrieve and verify the artifact through a trusted registry and bind its provenance. Passing this format check establishes neither authenticity nor availability.

Expected behavior or output

```
tag rejected; digest-shaped reference accepted for further verification
```

Concrete experiment

Try a 63-character digest and uppercase hexadecimal characters. Require the declared syntax policy, then use an actual registry digest only after independent retrieval verification.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: OCI-DESCRIPTORS. See the added references and revision index.

4.2 Implementing the Mechanism

4.2.1 Engineering model

Registries, Artifacts, and Software Supply Chains — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

4.2.2 Production verification

Registries, Artifacts, and Software Supply Chains verification —

Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Separate local tagging from a deliberate registry push

Provide explicit commands for publishing a built image without inventing credentials or a digest.

publish.sh

```
set -eu
: "${REGISTRY:?Set an authorized registry hostname}"
: "${REPOSITORY:?Set the repository path}"
: "${REVISION:?Set the source revision tag}"
: "${LOCAL_IMAGE:?Set an existing locally tested image}"
remote="$REGISTRY/$REPOSITORY:$REVISION"
docker tag "$LOCAL_IMAGE" "$remote"
docker push "$remote"
docker image inspect "$remote" --format '{{json .RepoDigests}}'
```

Run or inspect

```
sh publish.sh
```

Explanation

The required variables identify the publication destination and local artifact. Tagging changes local metadata; push performs the network write to the authorized registry.

RepoDigests can record registry content identity after push or pull. Authentication is managed outside the script, and no push was executed without Docker and registry authorization.

Expected behavior or output

On an authorized Docker/registry host: a successful push and JSON digest list for the published image. No digest is fabricated here.

Concrete experiment

Publish two different lab images under distinct revision tags and compare their digests. Do not overwrite a released revision tag instead of creating a new release.

Validation of this new example: gcc: target required

Technical references: DOCKER-PUSH. See the added references and revision index.

4.3 Verifying Production Behavior

4.3.1 Engineering model

Registries, Artifacts, and Software Supply Chains — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown,

file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

4.3.2 Production verification

Registries, Artifacts, and Software Supply Chains verification — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Check a release inventory before promotion

Reject an artifact inventory that omits a required runtime component.

example.py

```
import json
from pathlib import Path
manifest={'artifact':'harbor-lab','components':[{ 'name':'libstdc++','version':'record-from-build'},{'name':'libc','version':'record-from-build'}]}
Path('inventory.json').write_text(json.dumps(manifest,indent=2))
names={c['name'] for c in manifest['components']}
assert {'libstdc++','libc'}<=names
broken={'components':[manifest['components'][1]]}
assert not {'libstdc++','libc'}<={c['name'] for c in broken['components']}
print('complete inventory accepted; missing C++ runtime rejected')
```

Run or inspect

```
python3 example.py
```


Explanation

The check verifies component presence, not vulnerability status. Version strings are placeholders in this offline fixture; a release inventory needs measured versions and component identifiers.

An SBOM records what was built, not a guarantee that every component is safe. Scanner findings, provenance, licenses, and approved exceptions remain separate promotion inputs.

Expected behavior or output

complete inventory accepted; missing C++ runtime rejected

Concrete experiment

Add a rule rejecting placeholder versions and show that this teaching inventory fails it. Populate production versions from the build rather than guessing.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: SPDX. See the added references and revision index.

Worked example — Apply registries, artifacts, and software supply chains to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 5 — Kubernetes Architecture

This chapter develops **kubernetes architecture** as a production mechanism whose application and platform responsibilities can be tested and recovered.

5.1 Application Boundary

5.1.1 Engineering model

Kubernetes Architecture — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop

admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

5.1.2 Production verification

Kubernetes Architecture verification — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Keep desired replicas separate from observed ready replicas

An accepted desired state is not the same as a healthy service.

main.cpp

```
#include <iostream>

int main() {
    unsigned desired=3,created=3,ready=1;
    bool api_accepted=desired==created,service_capacity=ready>=2;
    std::cout<<std::boolalpha<<"objects-created="<<api_accepted<<"
    minimum-ready="<<service_capacity<<" ready="<<ready<< '/' <<desired<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The desired count is control-plane intent, while ready represents observed runtime capacity. Created objects do not guarantee useful instances when startup or readiness checks fail.

This model does not query Kubernetes. It demonstrates why application-level acceptance must follow API acceptance: the desired object and client-visible capacity are different boundaries.

Expected behavior or output

```
objects-created=true minimum-ready=false ready=1/3
```

Concrete experiment

Advance ready from 1 to 3 without changing desired state. Then model one unready instance during a rollout.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

5.2 Platform Boundary

5.2.1 Engineering model

Kubernetes Architecture — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

5.2.2 Production verification

Kubernetes Architecture verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Declare a Deployment whose selector matches its Pod template

Use controller-owned Pods rather than manually replicating individual processes.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
d=docs[0]
selector=d['spec']['selector']['matchLabels'];labels=d['spec']['template']['metadata']['labels']
assert all(labels.get(k)==v for k,v in selector.items())
assert d['spec']['replicas']==3
print('offline Deployment: selector matches template, desired replicas3')
```

example.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: harbor}
spec:
  replicas: 3
  selector:
    matchLabels: {app: harbor}
  template:
    metadata:
      labels: {app: harbor}
    spec:
      containers:
        - name: app
          image: harbor-service:lab
          imagePullPolicy: Never
          ports: [{name: http, containerPort: 8080}]
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

The Deployment controller reconciles replicas through ReplicaSets. Its selector identifies the workload's Pod labels, so the template must describe the intended managed set.

containerPort declares a port but does not start a listener, create a Service, or open a firewall. The application must listen on 8080 and the lab image must be loaded on the cluster.

Expected behavior or output

```
offline Deployment: selector matches template, desired replicas3
```

Concrete experiment

Change only the template label to other and make the offline check fail. Plan selector and label changes explicitly instead of editing them casually.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-DEPLOYMENT. See the added references and revision index.

5.3 Failure and Recovery

5.3.1 Engineering model

Kubernetes Architecture — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

5.3.2 Production verification

Kubernetes Architecture verification — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Detect that a controller has not observed the latest generation

Inspect a Kubernetes-shaped status fixture before declaring the latest configuration ready.

example.py

```
desired={'metadata':{'generation':8},'spec':{'replicas':3},'status':{'observedGeneration':7,'readyReplicas':3}}
def ready(d):return d['status'].get('observedGeneration',0)>=d['metadata']['generation'] and
d['status'].get('readyReplicas',0)>=d['spec']['replicas']
assert not ready(desired)
desired['status']['observedGeneration']=8
assert ready(desired)
print('old generation rejected; observed generation8 with3 ready accepted')
```

Run or inspect

```
python3 example.py
```

Explanation

Three ready replicas may still belong to the previous configuration. observedGeneration must catch up with generation before the fixture accepts the count as current evidence.

Production rollout checks also need updated and available replica counts, conditions, and application requests. This offline predicate is not a replacement for rollout status or end-to-end validation.

Expected behavior or output

```
old generation rejected; observed generation8 with3 ready accepted
```

Concrete experiment

Set readyReplicas to 2 after generation 8 is observed. Distinguish stale controller observation from insufficient ready capacity.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-DEPLOYMENT. See the added references and revision index.

Worked example — Apply kubernetes architecture to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>
```

```

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}

```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 6 — Pods and Workload Controllers

This chapter develops **pods and workload controllers** as a production mechanism whose application and platform responsibilities can be tested and recovered.

6.1 Defining the Contract

6.1.1 Engineering model

Pods and Workload Controllers — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop

admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

6.1.2 Production verification

Pods and Workload Controllers verification — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Choose a finite Job for one-shot work

Run a one-shot migration as finite batch work rather than restarting it indefinitely as a Deployment.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
j=docs[0]
assert j['kind']=='Job' and j['spec']['backoffLimit']==2
assert j['spec']['template']['spec']['restartPolicy']=='Never'
assert j['spec']['activeDeadlineSeconds']==120
print('offline Job: finite deadline120s and retry limit2')
```

example.yaml

```
apiVersion: batch/v1
kind: Job
metadata: {name: harbor-migrate}
spec:
  backoffLimit: 2
  activeDeadlineSeconds: 120
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: migrate
        image: harbor-migrate:lab
        imagePullPolicy: Never
        args: ["--once"]
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

Job completion depends on process outcome. restartPolicy Never leaves failed Pod attempts to the Job controller, while backoffLimit bounds the retry count.

Retried batch work must be idempotent or checkpointed: an effect can occur before the process reports success. The 120-second deadline bounds job lifetime, not transactional rollback.

Expected behavior or output

```
offline Job: finite deadline120s and retry limit2
```

Concrete experiment

Make a lab migration exit unsuccessfully after an idempotent step. Verify that a retry does not duplicate that effect and that deadline expiration is visible.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-JOB. See the added references and revision index.

6.2 Implementing the Mechanism

6.2.1 Engineering model

Pods and Workload Controllers — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

6.2.2 Production verification

Pods and Workload Controllers verification — Implementing the Mechanism

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume

unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Use StatefulSet identity with one claim per replica

Declare stable Pod identity and per-replica storage without claiming application replication.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
service, stateful=docs
assert service['spec']['clusterIP']=='None'
assert stateful['spec']['serviceName']==service['metadata']['name']
assert stateful['spec']['volumeClaimTemplates'][0]['metadata']['name']=='data'
print('offline StatefulSet: stable service identity and per-replica data claim')
```

example.yaml

```
apiVersion: v1
kind: Service
metadata: {name: harbor-stateful}
spec:
  clusterIP: None
  selector: {app: harbor-stateful}
  ports: [{name: app, port: 8080, targetPort: 8080}]
---
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: harbor-stateful}
spec:
  serviceName: harbor-stateful
  replicas: 2
  selector: {matchLabels: {app: harbor-stateful}}
  template:
    metadata: {labels: {app: harbor-stateful}}
    spec:
      containers:
        - name: app
          image: harbor-stateful:lab
          imagePullPolicy: Never
          volumeMounts: [{name: data, mountPath: /data}]
  volumeClaimTemplates:
    - metadata: {name: data}
      spec:
        accessModes: [ReadWriteOnce]
        resources: {requests: {storage: 1Gi}}
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

The headless Service and serviceName support StatefulSet identity. The claim template gives each replica its own persistent claim rather than pointing all writers at one ordinary file.

Stable names and volumes do not implement consensus, sharding, or backups. The application still defines coordination, and the cluster must supply suitable storage provisioning.

Expected behavior or output

offline StatefulSet: stable service identity and per-replica data claim

Concrete experiment

Scale a lab StatefulSet from two to three and inspect the distinct claims. Three claims are not proof of replicated application state.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-STATEFUL, K8S-STORAGE. See the added references and revision index.

6.3 Verifying Production Behavior

6.3.1 Engineering model

Pods and Workload Controllers — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

6.3.2 Production verification

Pods and Workload Controllers verification — Verifying Production Behavior

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets

incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Identify a replacement Pod by UID, not just its name

Use Pod UID to distinguish replacement from a new observation of the same object.

example.py

```
old={'name':'harbor-0','uid':'uid-old','restartCount':4}
new={'name':'harbor-0','uid':'uid-new','restartCount':0}
def same_instance(a,b):return a['uid']==b['uid']
assert old['name']==new['name'] and not same_instance(old,new)
print('same name, different UID: reset instance-local observations')
```

Run or inspect

```
python3 example.py
```

Explanation

A Pod name can be reused after deletion. UID identifies its API-object lifetime, so timers and counts from uid-old must not be attached to uid-new.

A container restart within the same Pod is another identity level. Monitoring must choose whether it tracks a Pod, container instance, or logical service rather than merging everything by name.

Expected behavior or output

```
same name, different UID: reset instance-local observations
```

Concrete experiment

Use the same UID with a higher restartCount. That represents a container restart observation, not necessarily Pod replacement.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-POD. See the added references and revision index.

Worked example — Apply pods and workload controllers to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 7 — Configuration and Secrets

This chapter develops **configuration and secrets** as a production mechanism whose application and platform responsibilities can be tested and recovered.

7.1 Application Boundary

7.1.1 Engineering model

Configuration and Secrets — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop

admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

7.1.2 Production verification

Configuration and Secrets verification — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Validate runtime configuration before applying it

Parse a bounded worker count without mutating the active configuration on failure.

main.cpp

```
#include <charconv>
#include <string_view>
#include <iostream>

int main() {
    unsigned workers=4;
    auto reload=[&](std::string_view s){unsigned
n=0;auto[p,e]=std::from_chars(s.data(),s.data()+s.size(),n);
    if(e!=std::errc{}||p!=s.data()+s.size()||n<1||n>32)return false;workers=n;return
true;};
    bool bad=reload("1000"),good=reload("8");
    std::cout<<"invalid="<<std::boolalpha<<bad<<" valid="<<good<<" workers="<<workers<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

ConfigMaps and environment variables deliver bytes, not validated application types. Complete parsing and range checks before publishing a new worker limit.

A bad reload should leave the last valid configuration active and emit a bounded error category. Resizing the real worker pool also needs lifecycle coordination; this fixture checks only the configuration decision.

Expected behavior or output

```
invalid=false valid=true workers=8
```

Concrete experiment

Try 8workers and 8x as inputs. Only the complete integer 8 should be accepted.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

7.2 Platform Boundary

7.2.1 Engineering model

Configuration and Secrets — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

7.2.2 Production verification

Configuration and Secrets verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Mount nonsecret configuration separately from a secret reference

Deliver ordinary configuration through a ConfigMap and reference credentials through a separately created Secret.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
config,pod=docs
assert config['data']['workers']=='4'
volume=pod['spec']['volumes'][0]
assert volume['secret']['secretName']=='harbor-credentials'
assert 'data' not in pod
print('offline wiring: ConfigMap workers and read-only Secret volume reference')
```

example.yaml

```
apiVersion: v1
kind: ConfigMap
metadata: {name: harbor-config}
data:
  workers: "4"
---
apiVersion: v1
kind: Pod
metadata: {name: harbor-config-reader}
spec:
  containers:
    - name: app
      image: harbor-service:lab
      imagePullPolicy: Never
      env:
        - name: WORKERS
          valueFrom: {configMapKeyRef: {name: harbor-config, key: workers}}
      volumeMounts:
        - {name: credentials, mountPath: /run/harbor-secrets, readOnly: true}
  volumes:
    - name: credentials
      secret: {secretName: harbor-credentials, defaultMode: 0400}
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

The manifest contains no secret value; it refers to a credential object by name. Configuration and credentials can therefore have distinct access and rotation policies.

Environment values are captured at startup. Mounted Secret changes have propagation and reload requirements, and Secret encoding alone is not encryption; storage and access controls remain necessary.

Expected behavior or output

```
offline wiring: ConfigMap workers and read-only Secret volume reference
```

Concrete experiment

Use dummy credentials in a disposable cluster and verify file access under the actual container UID. Never place production values in the manuscript or test logs.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-CONFIG, K8S-SECRET. See the added references and revision index.

7.3 Failure and Recovery

7.3.1 Engineering model

Configuration and Secrets — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation

must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

7.3.2 Production verification

Configuration and Secrets verification — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Tie a configuration rollout to an immutable content fingerprint

Compare exact configuration bytes with the fingerprint in a Pod-template annotation.

example.py

```
import hashlib
config=b'workers=4\ntimeout_ms=250\n'
first=hashlib.sha256(config).hexdigest()
changed=hashlib.sha256(config.replace(b'workers=4',b'workers=8')).hexdigest()
assert first!=changed
print('configuration change produces a different rollout fingerprint')
```

Run or inspect

```
python3 example.py
```

Explanation

Changing a content fingerprint in the Pod template changes the Deployment's desired template, allowing replacement processes to start with a specific configuration identity.

The hash is content identity, not authentication or secret protection. This fixture uses nonsecret configuration; publishing hashes of low-entropy secrets can also expose useful information to an attacker.

Expected behavior or output

```
configuration change produces a different rollout fingerprint
```

Concrete experiment

Reorder the configuration lines and observe the changed byte hash. Decide whether canonicalization belongs in the identity contract before using it in production.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-DEPLOYMENT, K8S-CONFIG. See the added references and revision index.

Worked example — Apply configuration and secrets to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 8 — Persistent Storage

This chapter develops **persistent storage** as a production mechanism whose application and platform responsibilities can be tested and recovered.

8.1 Defining the Contract

8.1.1 Engineering model

Persistent Storage — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

8.1.2 Production verification

Persistent Storage verification — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Use an explicit persistent-data directory instead of the working directory

Write state to an explicitly supplied data directory rather than assuming that the working directory is durable.

main.cpp

```
#include <filesystem>
#include <fstream>
#include <string>
#include <iostream>

int main() {
    const std::filesystem::path data="lab-volume";std::filesystem::create_directories(data);
    {std::ofstream out(data/"checkpoint.txt");out<<"generation=7\n";if(!out)return 1;}
    std::ifstream in(data/"checkpoint.txt");std::string line;if(!std::getline(in,line))return
2;
    std::cout<<"volume-state="<<line<<"\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The path input can map to a persistent-volume mount. The application does not assume that its current directory inside the image is writable or persistent.

This local write/read fixture does not establish persistence across Pod rescheduling. The platform must mount the right volume, and the application must define checkpoint and flush consistency before acknowledging durability.

Expected behavior or output

```
volume-state=generation=7
```

Concrete experiment

Use a new empty directory for the second read and require a missing-checkpoint outcome. A newly bound volume does not automatically contain old state.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

8.2 Implementing the Mechanism

8.2.1 Engineering model

Persistent Storage — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

8.2.2 Production verification

Persistent Storage verification — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Mount a PVC at the application's data boundary

Declare storage independently of the image without fabricating an available StorageClass.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
claim, pod=docs
assert
pod['spec']['volumes'][0]['persistentVolumeClaim']['claimName']==claim['metadata']['name']
assert pod['spec']['containers'][0]['volumeMounts'][0]['mountPath']=='/var/lib/harbor'
print('offline PVC mount: harbor-data -> /var/lib/harbor')
```

example.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: harbor-data}
spec:
  accessModes: [ReadWriteOnce]
  resources: {requests: {storage: 1Gi}}
---
apiVersion: v1
kind: Pod
metadata: {name: harbor-store}
spec:
  containers:
    - name: app
      image: harbor-service:lab
      imagePullPolicy: Never
      volumeMounts: [{name: data, mountPath: /var/lib/harbor}]
  volumes:
    - name: data
      persistentVolumeClaim: {claimName: harbor-data}
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

The claim states capacity and access mode; the Pod names the claim it mounts. Binding requires an appropriate provisioner or manually prepared volume in the target cluster.

ReadWriteOnce generally constrains read-write mounting by node, not a single application writer. It is neither database concurrency control nor a backup guarantee.

Expected behavior or output

```
offline PVC mount: harbor-data -> /var/lib/harbor
```

Concrete experiment

Observe Pending when no suitable storage is available. Classify that as a provisioning dependency failure rather than an application file-parsing defect.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-STORAGE. See the added references and revision index.

8.3 Verifying Production Behavior

8.3.1 Engineering model

Persistent Storage — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy,

degraded, and failed states from available evidence, the design is not operationally complete.

8.3.2 Production verification

Persistent Storage verification — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Require a restore check instead of trusting a backup filename

Restore an immutable local backup fixture and reject a corrupted copy.

example.py

```
from pathlib import Path
import hashlib,shutil
Path('state.bin').write_bytes(b'harbor-state-generation7')
expected=hashlib.sha256(Path('state.bin').read_bytes()).hexdigest()
shutil.copyfile('state.bin','backup.bin');shutil.copyfile('backup.bin','restored.bin')
assert hashlib.sha256(Path('restored.bin').read_bytes()).hexdigest()==expected
Path('restored.bin').write_bytes(b'corrupt')
assert hashlib.sha256(Path('restored.bin').read_bytes()).hexdigest()!=expected
print('restore verified; corrupted restore rejected')
```

Run or inspect

```
python3 example.py
```

Explanation

The checksum compares restored bytes with a trusted fixture identity. Finding a file named backup.bin is not sufficient evidence that a restore produced valid application data.

This file is immutable test data. Arbitrarily copying a live SQLite or other database file can miss a consistent state; use the database's supported backup or snapshot procedure instead.

Expected behavior or output

```
restore verified; corrupted restore rejected
```

Concrete experiment

After checking the checksum, parse and verify generation 7. Byte integrity and application-format compatibility are separate conditions.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-STORAGE, SQLITE-BACKUP. See the added references and revision index.

Worked example — Apply persistent storage to HarborDispatch**Requirement and design**

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
```

```

#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}

```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 9 — Kubernetes Networking and Services

This chapter develops **kubernetes networking and services** as a production mechanism whose application and platform responsibilities can be tested and recovered.

9.1 Application Boundary

9.1.1 Engineering model

Kubernetes Networking and Services — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation

must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

9.1.2 Production verification

Kubernetes Networking and Services verification — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Distinguish bind address from an advertised service endpoint

Distinguish the listener's wildcard bind address from the endpoint clients should use.

main.cpp

```
#include <string>
#include <iostream>

int main() {
    struct Endpoint{std::string host;unsigned port;};
    Endpoint bind{"0.0.0.0",8080},service{"harbor.default.svc.cluster.local",80};
    auto connectable=[](const Endpoint&e){return
!e.host.empty()&&e.host!="0.0.0.0"&&e.port>0&&e.port<=65535;};
    std::cout<<"bind-client-address="<<std::boolalpha<<connectable(bind)<<"
advertised="<<service.host<<':'<<service.port<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Binding 0.0.0.0 listens on local IPv4 interfaces. A Service name and port describe a client-facing route that can translate port 80 to the application's 8080.

The cluster.local suffix is an illustrative default, not universal cluster configuration. This fixture validates endpoint roles; it does not resolve DNS or establish network reachability.

Expected behavior or output

```
bind-client-address=false advertised=harbor.default.svc.cluster.local:80
```

Concrete experiment

Bind the application to 127.0.0.1 and explain why a Service normally cannot reach that listener from another network namespace.

Validation of this new example: gcc: passed

Technical references: K8S-SERVICE. See the added references and revision index.

9.2 Platform Boundary

9.2.1 Engineering model

Kubernetes Networking and Services — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

9.2.2 Production verification

Kubernetes Networking and Services verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Route a Service port to a named container port

Keep client port 80 separate from listener port 8080 and verify selector alignment.

example.py

```
import pathlib,yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
service,pod=docs
assert all(pod['metadata']['labels'].get(k)==v for k,v in service['spec']['selector'].items())
target=service['spec']['ports'][0]['targetPort']
ports=pod['spec']['containers'][0]['ports']
assert any(p['name']==target and p['containerPort']==8080 for p in ports)
print('offline route: Service80 -> named http -> container8080')
```

example.yaml

```
apiVersion: v1
kind: Service
metadata: {name: harbor}
spec:
  selector: {app: harbor}
  ports: [{name: web, port: 80, targetPort: http}]
---
apiVersion: v1
kind: Pod
metadata: {name: harbor-lab, labels: {app: harbor}}
spec:
  containers:
    - name: app
      image: harbor-service:lab
      imagePullPolicy: Never
      ports: [{name: http, containerPort: 8080}]
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

The Service selects Pods by label and resolves targetPort http through their named port. Clients use Service port 80 while the application still listens on 8080.

This manifest creates no public load balancer, Ingress, or TLS endpoint. A containerPort declaration does not create a socket; inspect endpoints and make real requests on the target cluster.

Expected behavior or output

```
offline route: Service80 -> named http -> container8080
```

Concrete experiment

Rename the Pod port to api without updating the Service. Make the offline assertion fail before a cluster deployment is attempted.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-SERVICE. See the added references and revision index.

9.3 Failure and Recovery

9.3.1 Engineering model

Kubernetes Networking and Services — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

9.3.2 Production verification

Kubernetes Networking and Services verification — Failure and Recovery

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Diagnose an empty ready-endpoint set from actual-shaped status data

Count ready endpoint addresses before blaming DNS or application code.

example.py

```
slices=[{'endpoints': [{'addresses': ['10.1.0.2'], 'conditions': {'ready': False}},
                      {'addresses': ['10.1.0.3'], 'conditions': {'ready': True}}]}]
ready=[a for s in slices for e in s['endpoints'] if e['conditions'].get('ready') is True for a
       in e['addresses']]
assert ready==['10.1.0.3']
print('ready endpoints1: inspect readiness before DNS when this count becomes0')
```

Run or inspect

```
python3 example.py
```

Explanation

The fixture distinguishes endpoint existence from readiness. An address appearing in an EndpointSlice is not automatically an eligible healthy backend.

With no ready endpoints, investigate labels, probes, and workload state. This is offline API-shaped data; actual inspection uses the Service label to select EndpointSlices on the target cluster.

Expected behavior or output

```
ready endpoints1: inspect readiness before DNS when this count becomes0
```

Concrete experiment

Mark the second endpoint unready and expect an empty result. Keep API readiness metadata separate from network reachability tests.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-ENDPOINTS. See the added references and revision index.

Worked example — Apply kubernetes networking and services to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }
```

```
int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 10 — Health, Readiness, and Lifecycle

This chapter develops **health, readiness, and lifecycle** as a production mechanism whose application and platform responsibilities can be tested and recovered.

10.1 Defining the Contract

10.1.1 Engineering model

Health, Readiness, and Lifecycle — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume

unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

10.1.2 Production verification

Health, Readiness, and Lifecycle verification — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Define three distinct probe predicates

Use initialization, internal progress, and admission state instead of one ambiguous healthy flag.

main.cpp

```
#include <iostream>

int main() {
    struct State{bool initialized=false;bool event_loop_progress=true;bool
draining=false;};State s;
    auto startup=[&]{return s.initialized;};auto live=[&]{return s.event_loop_progress;};auto
ready=[&]{return s.initialized&&!s.draining;};
    std::cout<<std::boolalpha<<startup()<<' '<<live()<<'
'<<ready()<<'\n';s.initialized=true;s.draining=true;
    std::cout<<startup()<<' '<<live()<<' '<<ready()<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Startup remains false until initialization finishes. Liveness observes internal progress rather than queue capacity. Readiness is false while initializing or draining.

These predicates are only as useful as the conditions they measure. A process can remain alive without useful progress, and an external outage does not always justify restarting it.

Expected behavior or output

```
false true false
true true false
```

Concrete experiment

Add overload as a readiness input without changing startup. Decide whether traffic removal or explicit load shedding is the correct service policy.

Validation of this new example: gcc: passed

Technical references: K8S-PROBES. See the added references and revision index.

10.2 Implementing the Mechanism

10.2.1 Engineering model

Health, Readiness, and Lifecycle — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

10.2.2 Production verification

Health, Readiness, and Lifecycle verification — Implementing the Mechanism

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Configure separate startup, liveness, and readiness endpoints

Align separate startup, liveness, and readiness routes with the application's distinct predicates.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
c=docs[0]['spec']['containers'][0]
assert {c[k]['httpGet']['path'] for k in
('startupProbe', 'livenessProbe', 'readinessProbe')}=={'/startup', '/live', '/ready'}
assert c['startupProbe']['failureThreshold']*c['startupProbe']['periodSeconds']==60
print('offline probes: three meanings; approximate startup allowance60s')
```

example.yaml

```
apiVersion: v1
kind: Pod
metadata: {name: harbor-probes}
spec:
  terminationGracePeriodSeconds: 20
  containers:
  - name: app
    image: harbor-service:lab
    imagePullPolicy: Never
    ports: [{name: http, containerPort: 8080}]
    startupProbe:
      httpGet: {path: /startup, port: http}
      periodSeconds: 2
      failureThreshold: 30
    livenessProbe:
      httpGet: {path: /live, port: http}
      periodSeconds: 5
      timeoutSeconds: 1
    readinessProbe:
      httpGet: {path: /ready, port: http}
      periodSeconds: 2
      timeoutSeconds: 1
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

The startup probe supplies an initialization failure budget before ordinary probe handling. The 60-second product is an approximate configuration allowance, not an exact wall-clock guarantee.

The application must implement all three HTTP paths and appropriate statuses. A manifest pointing to nonexistent routes fails regardless of whether the business code itself works.

Expected behavior or output

```
offline probes: three meanings; approximate startup allowance60s
```

Concrete experiment

On a lab target, delay initialization ten seconds and observe that readiness withholds traffic. The offline field check does not run that probe experiment.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-PROBES. See the added references and revision index.

10.3 Verifying Production Behavior

10.3.1 Engineering model

Health, Readiness, and Lifecycle — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

10.3.2 Production verification

Health, Readiness, and Lifecycle verification — Verifying Production Behavior

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume

unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Test SIGTERM-driven shutdown on the actual C++ process

Keep the signal handler minimal and let normal execution change readiness and terminate.

main.cpp

```
#include <signal>
#include <chrono>
#include <thread>
#include <iostream>
volatile std::sig_atomic_t stop=0;
extern "C" void request_stop(int){stop=1;}
int main(){if(std::signal(SIGTERM,request_stop)==SIG_ERR)return 1;
  std::cout<<"ready=1\n"<<std::flush;
  while(!stop)std::this_thread::sleep_for(std::chrono::milliseconds(5));
  std::cout<<"ready=0\ndrained\n";
}
```

test.py

```
import subprocess,selectors
subprocess.run(['g++','-std=c++20','-pthread','main.cpp','-o','app'],check=True)
p=subprocess.Popen(['./app'],stdout=subprocess.PIPE,stderr=subprocess.PIPE,text=True)
try:
    with selectors.DefaultSelector() as s:
        s.register(p.stdout,selectors.EVENT_READ)
        assert s.select(2),'startup timeout'
        assert p.stdout.readline()=='ready=1\n'
        p.terminate();out,err=p.communicate(timeout=2)
        assert p.returncode==0 and out=='ready=0\ndrained\n' and err==''
        print('SIGTERM observed: readiness removed and process exited0')
finally:
    if p.poll() is None:p.kill();p.communicate()
```

Run or inspect

```
python3 test.py
```

Explanation

The handler writes only volatile `sig_atomic_t` state. It does not log, allocate, acquire locks, or run complex cleanup from asynchronous signal context.

The parent waits for real startup output, sends SIGTERM, and requires bounded exit with readiness removed. This is a native process test, not validation of container PID 1 forwarding or Kubernetes grace-period enforcement.

Expected behavior or output

```
SIGTERM observed: readiness removed and process exited0
```

Concrete experiment

Add a 100 ms drain in ordinary execution after readiness is removed and confirm that exit still meets the two-second test limit.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: CPP20, K8S-PROBES. See the added references and revision index.

Worked example — Apply health, readiness, and lifecycle to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 11 — Resource Management

This chapter develops **resource management** as a production mechanism whose application and platform responsibilities can be tested and recovered.

11.1 Application Boundary

11.1.1 Engineering model

Resource Management — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop

admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

11.1.2 Production verification

Resource Management verification — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Reserve memory before admitting a batch

Reserve a byte budget before accepting a batch rather than depending on a container OOM kill.

main.cpp

```
#include <cstdint>
#include <iostream>

int main() {
    constexpr std::size_t budget=64*1024;std::size_t reserved=0;
    auto reserve=[&](std::size_t n){if(n>budget-reserved)return false;reserved+=n;return
true;};
    bool a=reserve(48*1024),b=reserve(24*1024);
    std::cout<<std::boolalpha<<"first="<<a<<" second="<<b<<" reserved-bytes="<<reserved<<"'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The reservation ledger rejects a request before allocating beyond the application budget. A container memory limit does not provide a graceful admission mechanism; exceeding it may terminate the process.

Leave headroom for stacks, allocator overhead, libraries, caches, and telemetry. The fixture tests accounting, not measured RSS or cgroup enforcement.

Expected behavior or output

```
first=true second=false reserved-bytes=49152
```

Concrete experiment

Add checked release and verify that rejected reservations consume no permits. Choose a budget below the process's total memory allowance.

Validation of this new example: gcc: passed

Technical references: K8S-RESOURCES, CPP20. See the added references and revision index.

11.2 Platform Boundary

11.2.1 Engineering model

Resource Management — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

11.2.2 Production verification

Resource Management verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Distinguish scheduling requests from runtime limits

Declare scheduling requests separately from runtime resource ceilings.

example.py

```
import pathlib,yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
r=docs[0]['spec']['containers'][0]['resources']
assert r['requests']['cpu']=='250m' and str(r['limits']['cpu'])=='1'
assert r['requests']['memory']=='64Mi' and r['limits']['memory']=='128Mi'
print('offline resource policy: request0.25CPU/64Mi, limit1CPU/128Mi')
```

example.yaml

```
apiVersion: v1
kind: Pod
metadata: {name: harbor-resources}
spec:
  containers:
  - name: app
    image: harbor-service:lab
    imagePullPolicy: Never
    resources:
      requests: {cpu: 250m, memory: 64Mi}
      limits: {cpu: "1", memory: 128Mi}
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

Requests help the scheduler place work; limits constrain runtime consumption under platform enforcement. CPU 250m means one quarter of a CPU unit, not 250 milliseconds per request.

A memory limit does not preallocate that quantity inside the C++ process. CPU limits may introduce throttling, and neither requests nor limits establish hard real-time isolation.

Expected behavior or output

```
offline resource policy: request0.25CPU/64Mi, limit1CPU/128Mi
```

Concrete experiment

Set CPU request equal to its limit and compare placement and contention on a lab cluster. Measure effects rather than inferring them from YAML alone.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-RESOURCES. See the added references and revision index.

11.3 Failure and Recovery

11.3.1 Engineering model

Resource Management — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

11.3.2 Production verification

Resource Management verification — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Classify an OOMKilled container from termination metadata

Use termination reason and supporting evidence instead of assuming that every exit 137 means OOM.

example.py

```
statuses=[{'reason':'OOMKilled','exitCode':137},{ 'reason':'Error','exitCode':137}]
def classify(s):return 'memory-limit investigation' if s.get('reason')== 'OOMKilled' else
'cause not established by exit code'
assert classify(statuses[0])!=classify(statuses[1])
for s in statuses:print(classify(s))
```

Run or inspect

```
python3 example.py
```

Explanation

The first record explicitly reports OOMKilled. The second supplies only exit 137, which can also result from other SIGKILL causes and is insufficient for that diagnosis.

Inspect memory use, limits, events, and previous container logs before raising the ceiling. A leak can eventually consume any larger limit, while an admission bug may require an application fix.

Expected behavior or output

```
memory-limit investigation
cause not established by exit code
```

Concrete experiment

Add reason Completed with exit code 0 and distinguish normal completion, resource termination, and deployment-driven shutdown.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-RESOURCES, K8S-POD. See the added references and revision index.

Worked example — Apply resource management to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }
```

```
int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 12 — Autoscaling

This chapter develops **autoscaling** as a production mechanism whose application and platform responsibilities can be tested and recovered.

12.1 Defining the Contract

12.1.1 Engineering model

Autoscaling — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

12.1.2 Production verification

Autoscaling verification — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Calculate a replica recommendation from an explicit utilization target

Apply the simplified HPA utilization ratio, then clamp the result to configured replica bounds.

main.cpp

```

#include <cmath>
#include <algorithm>
#include <iostream>

int main() {
    int current=3,min_replicas=2,max_replicas=8;double utilization=90,target=60;
    int raw=int(std::ceil(current*utilization/target));int
desired=std::clamp(raw,min_replicas,max_replicas);
    std::cout<<"raw="<<raw<<" bounded-desired="<<desired<<"\n";
}

```

Run or inspect

```

g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example

```

Explanation

Three replicas at 90% of requested CPU against a 60% target yield $\text{ceil}(3 \cdot 90 / 60)$, or five replicas. Utilization is relative to requests, not a machine-wide CPU percentage.

Actual HPA decisions also account for missing metrics, readiness, tolerance, stabilization, and scaling policies. The fixture teaches the central ratio rather than predicting the complete controller.

Expected behavior or output

```
raw=5 bounded-desired=5
```

Concrete experiment

Use utilization 240 and verify that the recommendation clamps at eight. Admission and backpressure must still work when the configured maximum cannot satisfy demand.

Validation of this new example: gcc: passed

Technical references: K8S-HPA. See the added references and revision index.

12.2 Implementing the Mechanism

12.2.1 Engineering model

Autoscaling — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

12.2.2 Production verification

Autoscaling verification — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Configure HPA with explicit bounds and downscale stabilization

Scale a Deployment using CPU utilization while preventing an immediate downscale after one brief dip.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
h=docs[0]['spec']
assert h['minReplicas']==2 and h['maxReplicas']==8
assert h['metrics'][0]['resource']['target']['averageUtilization']==60
assert h['behavior']['scaleDown']['stabilizationWindowSeconds']==300
print('offline HPA: 2..8 replicas, CPU target60%, downscale window300s')
```

example.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: harbor}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: harbor}
  minReplicas: 2
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: {type: Utilization, averageUtilization: 60}
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

The scale target names an existing Deployment. CPU utilization needs defined CPU requests and metrics infrastructure; the HPA manifest does not supply either automatically.

The downscale window stabilizes recommendation history. It does not make scale-up instantaneous, guarantee startup, or solve downstream database capacity. The offline check verifies configuration fields only.

Expected behavior or output

```
offline HPA: 2..8 replicas, CPU target60%, downscale window300s
```

Concrete experiment

Remove the Deployment's CPU request in a lab and inspect HPA conditions. Missing metrics must not be interpreted as low load.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-HPA. See the added references and revision index.

12.3 Verifying Production Behavior

12.3.1 Engineering model

Autoscaling — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

12.3.2 Production verification

Autoscaling verification — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Demonstrate why a downscale window prevents flapping

Retain the highest recent recommendation before accepting a smaller replica count.

main.cpp

```
#include <deque>
#include <algorithm>
#include <iostream>
#include <utility>

int main() {
    std::deque<std::pair<int,int>> recommendations;
    auto stabilized=[&](int time,int desired){recommendations.emplace_back(time,desired);
        while(!recommendations.empty()&&recommendations.front().first<time-300)recommendations
        .pop_front();
        int result=desired;for(auto[t,n]:recommendations)result=std::max(result,n);return
        result;};
    std::cout<<stabilized(0,6)<<' '<<stabilized(10,2)<<' '<<stabilized(301,2)<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The recommendation of six remains in the 300-second window when demand briefly falls to two at second 10. At second 301, that older recommendation has expired.

This is a simplified downscale-history model, not the complete Kubernetes controller. It explains why inspecting one instantaneous metric is insufficient to reconstruct a scaling decision.

Expected behavior or output

```
6 6 2
```

Concrete experiment

Add a recommendation of five at second 200. The service should not downscale to two at second 301 because that newer recommendation remains in the window.

Validation of this new example: gcc: passed

Technical references: K8S-HPA. See the added references and revision index.

Worked example — Apply autoscaling to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
```

```

#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}

```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 13 — Production Service Discovery

This chapter develops **production service discovery** as a production mechanism whose application and platform responsibilities can be tested and recovered.

13.1 Application Boundary

13.1.1 Engineering model

Production Service Discovery — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop

admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

13.1.2 Production verification

Production Service Discovery verification — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Keep endpoint discovery separate from connection ownership

Give each operation an immutable endpoint snapshot so discovery refresh does not mutate its destination.

main.cpp

```
#include <memory>
#include <vector>
#include <string>
#include <iostream>

int main() {
    auto current=std::make_shared<const
std::vector<std::string>>(std::vector<std::string>{"worker-a:8080"});
    auto operation=current;current=std::make_shared<const
std::vector<std::string>>(std::vector<std::string>{"worker-b:8080"});
    std::cout<<"in-flight="<<operation->front()<<" new-request="<<current->front()<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The operation owns its snapshot independently of the currently published membership vector. Refreshing current therefore does not invalidate the old vector while an operation still uses it.

This does not keep worker-a alive or authorize unlimited retries to a retired instance. Deadlines, health, state ownership, and generation checks remain connection-level policy.

Expected behavior or output

```
in-flight=worker-a:8080 new-request=worker-b:8080
```

Concrete experiment

Remove every endpoint from the new snapshot. Fresh requests should report unavailable, while existing operations complete or cancel under their original deadlines.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

13.2 Platform Boundary

13.2.1 Engineering model

Production Service Discovery — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

13.2.2 Production verification

Production Service Discovery verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Use headless discovery only when the client owns endpoint selection

Use a headless Service only when the client owns endpoint selection and refresh.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
s=docs[0]['spec']
assert s['clusterIP']=='None' and s['publishNotReadyAddresses'] is False
assert s['selector']=={'app':'harbor-worker'}
print('offline headless service: client resolves worker endpoints')
```

example.yaml

```
apiVersion: v1
kind: Service
metadata: {name: harbor-workers}
spec:
  clusterIP: None
  publishNotReadyAddresses: false
  selector: {app: harbor-worker}
  ports: [{name: rpc, port: 8080, targetPort: 8080}]
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

clusterIP None avoids allocating the usual virtual Service IP. Discovery can expose backing addresses, leaving the client responsible for selection, failures, and membership refresh.

publishNotReadyAddresses false does not request publication of unready addresses, but actual DNS records depend on cluster behavior. Client-side discovery is a different contract, not automatically a better balancing strategy.

Expected behavior or output

```
offline headless service: client resolves worker endpoints
```

Concrete experiment

Compare DNS answers for a ClusterIP Service and a headless Service in a lab. Record TTL behavior without treating one answer as permanent membership.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-DNS, K8S-SERVICE. See the added references and revision index.

13.3 Failure and Recovery

13.3.1 Engineering model

Production Service Discovery — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

13.3.2 Production verification

Production Service Discovery verification — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Reject a stale DNS-shaped cache entry after its local expiry

Expire a DNS-shaped cache entry locally instead of silently extending freshness after a failed refresh.

main.cpp

```
#include <optional>
#include <string>
#include <iostream>

int main() {
    struct Entry{std::string address;int expires;};std::optional<Entry>
    cached=Entry{"10.0.0.2",100};
    auto resolve=[&](int now,bool
    refresh_ok)->std::optional<std::string>{if(cached&&now<cached->expires)return cached->address;
    if(!refresh_ok)return{};cached=Entry{"10.0.0.3",now+100};return cached->address;};
    auto a=resolve(99,false),b=resolve(101,false),c=resolve(102,true);
    std::cout<<"before="<<a<<" expired-unavailable="<<std::boolalpha<<!b<<"
    refreshed="<<*c<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The policy does not renew expiry merely because discovery failed. Serving stale endpoints may be valid for some applications, but it requires a separately bounded stale-on-error contract.

Expiry uses virtual time in this fixture, not a real resolver. Production clients must also handle failure before TTL expiry and prevent synchronized refresh or retry bursts.

Expected behavior or output

```
before=10.0.0.2 expired-unavailable=true refreshed=10.0.0.3
```

Concrete experiment

Add a ten-unit stale grace period and label returned entries stale. Do not give them the same freshness metadata as a valid cache entry.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Apply production service discovery to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
```

```

#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}

```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 14 — TLS, Certificates, and mTLS

This chapter develops **tls**, **certificates**, and **mtls** as a production mechanism whose application and platform responsibilities can be tested and recovered.

14.1 Defining the Contract

14.1.1 Engineering model

TLS, Certificates, and mTLS — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop

admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

14.1.2 Production verification

TLS, Certificates, and mTLS verification — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Configure peer verification and expected hostname on a TLS client object

Use OpenSSL client configuration APIs without treating object creation as a completed handshake.

main.cpp

```
#include <openssl/ssl.h>
#include <memory>
#include <iostream>

int main() {
    std::unique_ptr<SSL_CTX, decltype(&SSL_CTX_free)>
ctx(SSL_CTX_new(TLS_client_method()), SSL_CTX_free);
    if(!ctx) return 1; SSL_CTX_set_verify(ctx.get(), SSL_VERIFY_PEER, nullptr);
    if(SSL_CTX_set_min_proto_version(ctx.get(), TLS1_2_VERSION) != 1) return 2;
    std::unique_ptr<SSL, decltype(&SSL_free)> ssl(SSL_new(ctx.get()), SSL_free); if(!ssl) return
3;
    if(SSL_set1_host(ssl.get(), "worker.test") != 1 || SSL_set_tlsext_host_name(ssl.get(), "worker.t
est") != 1) return 4;
    if(SSL_get_verify_mode(ssl.get()) != SSL_VERIFY_PEER) return 5;
    std::cout<<"peer verification and worker.test identity configured; no handshake
performed\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example -lssl -lcrypto
./example
```

Explanation

SSL_VERIFY_PEER enables chain-verification policy, and SSL_set1_host sets the expected service identity. SNI supplies a virtual-host name to the server; it does not replace identity verification.

The context still needs trusted anchors before connecting. No SSL_connect call occurs, so the test claims configuration only. RAII releases the SSL connection object before its context.

Expected behavior or output

```
peer verification and worker.test identity configured; no handshake performed
```

Concrete experiment

Load an explicit lab CA with SSL_CTX_load_verify_locations, then add separate trusted-name and wrong-name handshake tests. Never disable verification just to force a connection.

Validation of this new example: gcc: passed

Technical references: OPENSSESSL-HOST, TLS13. See the added references and revision index.

14.2 Implementing the Mechanism

14.2.1 Engineering model

TLS, Certificates, and mTLS — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally

complete.

14.2.2 Production verification

TLS, Certificates, and mTLS verification — Implementing the Mechanism

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Reference a TLS certificate and key through a read-only Secret volume

Reference certificate files through a read-only Secret volume while keeping private-key bytes out of the manifest.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
p=docs[0]['spec'];c=p['containers'][0]
assert c['args'][1]=='/run/tls/tls.crt' and c['args'][3]=='/run/tls/tls.key'
assert c['volumeMounts'][0]['readOnly'] is True
assert p['volumes'][0]['secret']['secretName']=='harbor-server-tls'
print('offline TLS mount: certificate/key paths, no embedded private key')
```

example.yaml

```
apiVersion: v1
kind: Pod
metadata: {name: harbor-tls}
spec:
  containers:
    - name: app
      image: harbor-tls:lab
      imagePullPolicy: Never
      args: ["--cert", "/run/tls/tls.crt", "--key", "/run/tls/tls.key"]
      volumeMounts: [{name: tls, mountPath: /run/tls, readOnly: true}]
  volumes:
    - name: tls
      secret: {secretName: harbor-server-tls, defaultMode: 0400}
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

The Secret must be created through an authorized credential workflow. The read-only mount prevents this container from modifying those files, but access under the actual UID still needs testing.

Updating a Secret is not the same as reloading a TLS context. The application must specify when new connections use the rotated certificate and how existing sessions finish.

Expected behavior or output

offline TLS mount: certificate/key paths, no embedded private key

Concrete experiment

Rotate a dummy certificate in a lab. Check new handshakes separately from existing connections instead of restarting every replica simultaneously.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-SECRET, TLS13. See the added references and revision index.

14.3 Verifying Production Behavior

14.3.1 Engineering model

TLS, Certificates, and mTLS — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally

complete.

14.3.2 Production verification

TLS, Certificates, and mTLS verification — Verifying Production Behavior

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Check certificate expiration before it becomes an outage

Use OpenSSL checkend exit status to evaluate two certificate-expiry warning windows.

check.sh

```

set -eu
openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem -out cert.pem -days 1 -subj
/CN=lab.test 2>generation.log
openssl x509 -in cert.pem -noout -checkend 0 >now.txt
if openssl x509 -in cert.pem -noout -checkend 172800 >later.txt; then exit 1; fi
printf 'valid now; expiration occurs within48h\n'
```

Run or inspect

```
sh check.sh
```

Explanation

checkend 0 asks whether expiration is already reached; checkend 172800 evaluates a 48-hour window. A one-day lab certificate passes the first test but fails the second.

An expiry check does not verify trust, service name, revocation, or matching private key. It supplies one operational signal, and the generated credentials are disposable lab material.

Expected behavior or output

```
valid now; expiration occurs within48h
```

Concrete experiment

Generate a three-day certificate and expect both windows to pass. Keep chain and hostname verification as independent tests.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: OPENSsl-X509. See the added references and revision index.

Worked example — Apply tls, certificates, and mtls to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```

#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
```

```
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 15 — Service Mesh Architecture

This chapter develops **service mesh architecture** as a production mechanism whose application and platform responsibilities can be tested and recovered.

15.1 Application Boundary

15.1.1 Engineering model

Service Mesh Architecture — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop

admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

15.1.2 Production verification

Service Mesh Architecture verification — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Prevent retry multiplication across application and mesh layers

Calculate downstream attempt multiplication when application and mesh retries are both enabled.

main.cpp

```
#include <iostream>

int main() {
    unsigned app_attempts=3,mesh_attempts_per_call=3;
    std::cout<<"uncoordinated maximum="<<app_attempts*mesh_attempts_per_call<<'\n';
    app_attempts=1;std::cout<<"single retry owner
maximum="<<app_attempts*mesh_attempts_per_call<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Three application attempts can each trigger three mesh attempts, producing nine backend calls for one operation. Retry layers can therefore amplify the load during an outage.

Choosing one retry owner reduces multiplication but does not make a mutation safe to repeat. The operation still needs idempotency, a shared deadline, and a retryable-error policy.

Expected behavior or output

```
uncoordinated maximum=9
single retry owner maximum=3
```

Concrete experiment

Add a 100 ms logical deadline and one total attempt budget across both layers. Do not grant every retry a fresh full deadline.

Validation of this new example: gcc: passed

Technical references: ISTIO-TRAFFIC. See the added references and revision index.

15.2 Platform Boundary

15.2.1 Engineering model

Service Mesh Architecture — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

15.2.2 Production verification

Service Mesh Architecture verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Require mesh mTLS through an explicit PeerAuthentication policy

Declare Istio PeerAuthentication rather than assuming that an ordinary cluster socket is authenticated.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
p=docs[0]
assert p['apiVersion']=='security.istio.io/v1'
assert p['spec']['mtls']['mode']=='STRICT'
assert p['spec']['selector']['matchLabels']=='app: harbor'
print('offline Istio policy: selected harbor workloads require STRICT mTLS')
```

example.yaml

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: harbor-strict
  namespace: default
spec:
  selector:
    matchLabels: {app: harbor}
  mtls:
    mode: STRICT
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

The custom resource affects selected workloads only under a compatible installed Istio control and data plane. It does not enforce mTLS on a plain cluster without that implementation.

mTLS establishes transport peer identity under the mesh authority, while application authorization decides which identity may invoke which operation. Strict mode can reject plaintext clients and must be tested before cutover.

Expected behavior or output

```
offline Istio policy: selected harbor workloads require STRICT mTLS
```

Concrete experiment

On an isolated Istio cluster, compare a permitted mesh client with a plaintext client. Offline YAML parsing does not verify transport enforcement.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: ISTIO-MTLS. See the added references and revision index.

15.3 Failure and Recovery

15.3.1 Engineering model

Service Mesh Architecture — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

15.3.2 Production verification

Service Mesh Architecture verification — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Keep proxy readiness and application readiness distinct

Evaluate proxy and application readiness separately before reporting the request path ready.

example.py

```
statuses=[{'name':'app','ready':True},{'name':'proxy','ready':False}]
ready=all(c['ready'] for c in statuses)
assert not ready
print('application ready but proxy unready: combined readiness false')
```

Run or inspect

```
python3 example.py
```

Explanation

The data-plane proxy can block traffic while the application itself is healthy. Combined readiness must include every component required by the actual request path.

Restarting C++ code will not necessarily repair a broken proxy policy. Inspect per-container status, logs, and configuration generation before choosing a mitigation.

Expected behavior or output

```
application ready but proxy unready: combined readiness false
```

Concrete experiment

Set app readiness false and proxy readiness true. The combined result remains false, but the diagnostic category should identify the application rather than the proxy.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-POD, ISTIO-MTLS. See the added references and revision index.

Worked example — Apply service mesh architecture to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
```

```

std::signal(SIGTERM, request_stop);
std::signal(SIGINT, request_stop);
std::cout << "ready=1\n";
while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
std::cout << "ready=0 draining=1\n";
std::cout << "shutdown=complete\n";
}

```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 16 — Metrics and Instrumentation

This chapter develops **metrics and instrumentation** as a production mechanism whose application and platform responsibilities can be tested and recovered.

16.1 Defining the Contract

16.1.1 Engineering model

Metrics and Instrumentation — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop

admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

16.1.2 Production verification

Metrics and Instrumentation verification — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Define metric units and outcome labels at the request boundary

Emit counters with a finite outcome dimension instead of request IDs as metric labels.

main.cpp

```
#include <array>
#include <iostream>
#include <string_view>

int main() {
    const std::array<std::string_view,3>
    outcomes{"ok","invalid","busy"};std::array<unsigned,3> counts{2,1,0};
    for(std::size_t i=0;i<outcomes.size();++i)
        std::cout<<"harbor_requests_total{outcome=\""<<outcomes[i]<<"\"} "<<counts[i]<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The `_total` suffix denotes a cumulative count rather than a rate. The three known outcome values bound label cardinality independently of request volume.

This fixture prints Prometheus-style text without running an HTTP exporter or Prometheus server. A production exporter also needs escaping, reset behavior, and a declared logical counting boundary.

Expected behavior or output

```
harbor_requests_total{outcome="ok"} 2
harbor_requests_total{outcome="invalid"} 1
harbor_requests_total{outcome="busy"} 0
```

Concrete experiment

Add a retry for an existing job and decide whether the metric counts attempts or operations. Use separate names and definitions rather than mixing both meanings.

Validation of this new example: gcc: passed

Technical references: PROM-NAMING. See the added references and revision index.

16.2 Implementing the Mechanism

16.2.1 Engineering model

Metrics and Instrumentation — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

16.2.2 Production verification

Metrics and Instrumentation verification — Implementing the Mechanism

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Accumulate counter, gauge, and cumulative latency buckets

Keep completed-request count, in-flight state, and cumulative duration buckets distinct.

main.cpp

```
#include <array>
#include <iostream>

int main() {
    unsigned requests=0,in_flight=0;std::array<unsigned,3> buckets{};double sum=0;
    const std::array<double,3> bounds{0.01,0.1,1.0};
    for(double seconds:std::array{0.005,0.050,0.200}){++in_flight;++requests;sum+=seconds;
        for(std::size_t
i=0;i<bounds.size();++i)if(seconds<=bounds[i])++buckets[i];--in_flight;}
    std::cout<<"requests="<<requests<<" in-flight="<<in_flight<<"
buckets="<<buckets[0]<<','<<buckets[1]<<','<<buckets[2]<<" sum_s="<<sum<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The request counter grows while `in_flight` returns to zero after completion. Each duration contributes to every inclusive upper-bound bucket it fits, producing cumulative counts of one, two, and three.

The bounds and sum use seconds. `sum/count` loses the distribution, while buckets preserve threshold information but not exact tail values. Concurrent aggregation needs synchronization or appropriately merged shards.

Expected behavior or output

```
requests=3 in-flight=0 buckets=1,2,3 sum_s=0.255
```

Concrete experiment

Add a two-second request and retain an infinite bucket or total count for values above the largest finite boundary.

Validation of this new example: gcc: passed

Technical references: PROM-HISTOGRAMS. See the added references and revision index.

16.3 Verifying Production Behavior

16.3.1 Engineering model

Metrics and Instrumentation — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

16.3.2 Production verification

Metrics and Instrumentation verification — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown,

file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Measure label-series growth before deploying a metric

Calculate metric-series growth before adding an unbounded user dimension.

main.cpp

```
#include <stdint>
#include <iostream>

int main() {
    std::uint64_t services=5,outcomes=4,regions=3,users=100000;
    auto bounded=services*outcomes*regions;
    std::cout<<"bounded-series="<<bounded<<" with-user-id="<<bounded*users<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The bounded label set permits sixty combinations. Adding 100,000 user IDs raises the potential space to six million even before histogram buckets or more instances are considered.

Not every combination must exist for cardinality to become expensive.

High-cardinality identifiers belong in controlled logs or traces rather than unrestricted aggregate metric labels.

Expected behavior or output

```
bounded-series=60 with-user-id=6000000
```

Concrete experiment

Add eleven histogram buckets plus count and sum. Estimate thirteen series per label set and compare the total with the telemetry budget.

Validation of this new example: gcc: passed

Technical references: PROM-NAMING. See the added references and revision index.

Worked example — Apply metrics and instrumentation to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 17 — OpenTelemetry

This chapter develops **opentelemetry** as a production mechanism whose application and platform responsibilities can be tested and recovered.

17.1 Application Boundary

17.1.1 Engineering model

OpenTelemetry — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

17.1.2 Production verification

OpenTelemetry verification — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Create and end an actual OpenTelemetry C++ span

Create and end an actual OpenTelemetry C++ span using an ostream exporter.

main.cpp

```
#include <opentelemetry/sdk/trace/tracer_provider.h>
#include <opentelemetry/sdk/trace/simple_processor.h>
#include <opentelemetry/exporters/ostream/span_exporter.h>
#include <memory>
int main(){
    namespace sdk=opentelemetry::sdk::trace;
    auto exporter=std::make_unique<opentelemetry::exporter::trace::OStreamSpanExporter>();
    auto processor=std::make_unique<sdk::SimpleSpanProcessor>(std::move(exporter));
    auto provider=std::make_shared<sdk::TracerProvider>(std::move(processor));
    auto tracer=provider->GetTracer("harbor.section17","1.0");
    auto span=tracer->StartSpan("job.process");
    span->SetAttribute("job.kind","square");
    span->SetStatus(opentelemetry::trace::StatusCode::kOk);
    span->End();provider->ForceFlush();
}
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(OtelSection LANGUAGES CXX)
find_package(opentelemetry-cpp CONFIG REQUIRED)
add_executable(example main.cpp)
target_compile_features(example PRIVATE cxx_std_20)
target_link_libraries(example PRIVATE opentelemetry-cpp::trace
opentelemetry-cpp::ostream_span_exporter)
```

Run or inspect

```
cmake -S . -B build -DCMAKE_PREFIX_PATH=/path/to/installed/opentelemetry
cmake --build build
./build/example
```

Explanation

SimpleSpanProcessor forwards an ended span to its exporter. The provider owns the processor and exporter, while the tracer identifies the instrumentation scope. The job-kind attribute is bounded categorical data.

End completes the span and ForceFlush requests export completion before shutdown. This is real SDK integration source, but the OpenTelemetry C++ SDK was unavailable here, so compilation and execution are explicitly unverified.

Expected behavior or output

With a compatible installed OpenTelemetry C++ SDK: one exported job.process span with job.kind=square and OK status; generated IDs/timestamps vary.

Concrete experiment

Create a parent request span and a child processing span with explicit context. Inspect parent and trace IDs in exported output rather than assuming lexical nesting creates the relationship.

Validation of this new example: gcc: target required

Technical references: OTEL-CPP. See the added references and revision index.

17.2 Platform Boundary

17.2.1 Engineering model

OpenTelemetry — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

17.2.2 Production verification

OpenTelemetry verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Wire an OTLP receiver through processors to an exporter

Connect a local Collector receiver, processors, and exporter through an explicit trace pipeline.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
c=docs[0];pipeline=c['service']['pipelines']['traces']
assert pipeline['processors']=='memory_limiter','batch'
for kind in ('receivers','processors','exporters'):
    assert all(name in c[kind] for name in pipeline[kind])
print('offline Collector: OTLP -> memory_limiter -> batch -> debug')
```

example.yaml

```
receivers:
  otlp:
    protocols:
      grpc: {endpoint: "127.0.0.1:4317"}
processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 128
    spike_limit_mib: 32
  batch:
    timeout: 1s
    send_batch_size: 128
exporters:
  debug:
    verbosity: basic
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [debug]
```

Run or inspect

```
python3 example.py
# With a compatible Collector binary:
otelcol validate --config=example.yaml
```

Explanation

Top-level component definitions are inactive until referenced by a service pipeline. The memory limiter precedes batching to help control pressure before batching adds queued state.

The debug exporter is an inspection destination, not durable storage. A loopback receiver avoids opening an unauthenticated public port; the installed Collector distribution must still provide the named components.

Expected behavior or output

```
offline Collector: OTLP -> memory_limiter -> batch -> debug
```

Concrete experiment

Remove batch from the pipeline while retaining its definition. It should no longer process spans; run the installed Collector's configuration validation separately.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: OTEL-COLLECTOR. See the added references and revision index.

17.3 Failure and Recovery

17.3.1 Engineering model

OpenTelemetry — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

17.3.2 Production verification

OpenTelemetry verification — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Drop telemetry predictably when an exporter cannot keep up

Reject excess telemetry and count the drop rather than growing an exporter backlog without limit.

main.cpp

```
#include <deque>
#include <string>
#include <iostream>

int main() {
    std::deque<std::string> pending; unsigned dropped=0;
    auto emit=[&](std::string event){if(pending.size()==2){++dropped;return
false;}pending.push_back(std::move(event));return true;};
    emit("span1");emit("span2");bool third=emit("span3");
    std::cout<<"queued="<<pending.size()<<" dropped="<<dropped<<"
third="<<std::boolalpha<<third<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The third span is rejected before queue growth. A separate dropped counter records evidence loss without recursively emitting another event into the same full queue.

The queue is bounded by event count, not by total string bytes. Production events also need payload and attribute limits; exporter failure must not create unbounded blocking or crash the primary application.

Expected behavior or output

```
queued=2 dropped=1 third=false
```

Concrete experiment

Add a 32-byte event limit and test oversized payload rejection separately from queue-full rejection. Report categories without logging the rejected raw payload.

Validation of this new example: gcc: passed

Technical references: OTEL-COLLECTOR. See the added references and revision index.

Worked example — Apply opentelemetry to HarborDispatch**Requirement and design**

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```

#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}

```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 18 — Distributed Tracing

This chapter develops **distributed tracing** as a production mechanism whose application and platform responsibilities can be tested and recovered.

18.1 Defining the Contract

18.1.1 Engineering model

Distributed Tracing — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

18.1.2 Production verification

Distributed Tracing verification — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally

complete.

NEW SECTION EXAMPLE

Validate a traceparent header's fixed-width structure

Validate the fixed-width structure of a W3C version-00 traceparent fixture.

main.cpp

```
#include <string_view>
#include <algorithm>
#include <iostream>

int main() {
    auto hex=[](std::string_view s){return std::all_of(s.begin(),s.end(),[](char
c){return(c>='0'&&c<='9')|| (c>='a'&&c<='f');}));};
    auto valid=[&](std::string_view
s){if(s.size()!=55||s.substr(0,3)!="00-"||s[35]!='-'||s[52]!='-')return false;
        auto trace=s.substr(3,32),parent=s.substr(36,16);return hex(trace)&&hex(parent)&&hex(s
.substr(53,2))&&trace.find_first_not_of('0')!=trace.npos&&parent.find_first_not_of('0')!=paren
t.npos;};
    std::string_view good="00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01";
    std::cout<<std::boolalpha<<"valid="<<valid(good)<<"
truncated="<<valid(good.substr(0,54))<<"\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The 32-hex trace ID and 16-hex parent ID occupy fixed positions and cannot be all zero. The final two hexadecimal characters encode trace flags; parsing this metadata does not authenticate the caller.

This strict fixture handles version 00 only. A production propagator needs the specification's version and forward-compatibility rules; trace identifiers are correlation data, never authorization tokens.

Expected behavior or output

```
valid=true truncated=false
```

Concrete experiment

Replace the trace ID with 32 zeros and require rejection. Handle future versions explicitly rather than accepting arbitrary-length strings.

Validation of this new example: gcc: passed

Technical references: W3C-TRACE. See the added references and revision index.

18.2 Implementing the Mechanism

18.2.1 Engineering model

Distributed Tracing — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

18.2.2 Production verification

Distributed Tracing verification — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Pass trace context explicitly through a worker future

Copy trace context into asynchronous work instead of assuming thread-local state follows a future.

main.cpp

```
#include <future>
#include <string>
#include <iostream>

int main() {
    struct Context{std::string trace,span;};Context parent{"trace7","request"};
    auto job=std::async(std::launch::async,[parent]{return parent.trace+
parent="+parent.span+" child=worker";});
    std::cout<<job.get()<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The lambda owns a Context copy, preserving its values on another thread and after changes to the caller's local object. The worker uses a new span identity that refers to its parent.

This string fixture illustrates propagation ownership, not a complete OpenTelemetry encoding. A real adapter must capture, attach, and restore context when using a thread pool.

Expected behavior or output

```
trace7 parent=request child=worker
```

Concrete experiment

Modify the caller's parent span after launching work and verify that the captured value remains unchanged. Add a context-restoration scope in a real worker pool.

Validation of this new example: gcc: passed

Technical references: OTEL-CPP, W3C-TRACE. See the added references and revision index.

18.3 Verifying Production Behavior

18.3.1 Engineering model

Distributed Tracing — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown,

file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

18.3.2 Production verification

Distributed Tracing verification — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Find critical-path duration without summing overlapping spans

Distinguish request elapsed time from the sum of overlapping child spans.

main.cpp

```
#include <array>
#include <algorithm>
#include <iostream>

int main() {
    struct Span{int start,end;};const std::array children{Span{10,50},Span{20,60}};int sum=0;
    for(auto s:children)sum+=s.end-s.start;
    std::cout<<"child-duration-sum="<<sum<<" parent-elapsed="<<60-0<<" overlap="<<50-20<<"\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Each child lasts forty time units, with thirty units of overlap. Their sum is eighty, but the declared parent interval from zero to sixty lasts sixty.

Across machines, clock uncertainty adds another complication. Critical-path analysis needs causal parentage and waiting dependencies, not uncritical addition of every span duration.

Expected behavior or output

```
child-duration-sum=80 parent-elapsed=60 overlap=30
```

Concrete experiment

Move the second child to 50–90 and the parent end to 90. Child work still totals eighty, but request elapsed time increases because overlap was removed.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Apply distributed tracing to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 19 — Centralized Logging

This chapter develops **centralized logging** as a production mechanism whose application and platform responsibilities can be tested and recovered.

19.1 Application Boundary

19.1.1 Engineering model

Centralized Logging — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

19.1.2 Production verification

Centralized Logging verification — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally

complete.

NEW SECTION EXAMPLE

Escape structured log strings and exclude the token field

Escape an untrusted request label into one JSON log record and omit the token field.

main.cpp

```
#include <string>
#include <string_view>
#include <sstream>
#include <iomanip>
#include <iostream>

int main() {
    auto quote=[](std::string_view s){std::ostringstream out;out<<'";for(unsigned char c:s){
        if(c=='"'||c=='\\')out<<'\\'<<char(c);else
if(c<0x20)out<<"\\u"<<std::hex<<std::setw(4)<<std::setfill('0')<<unsigned(c);else
out<<char(c);}out<<'";return out.str();};
    std::string request="job\n7";std::string token="not-for-logs";(void)token;
    std::cout<<{"event\":\"rejected\", \"request\": "<<quote(request)<<\", \"reason\": \"syntax\"
}\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The escaper encodes control characters rather than writing raw newlines into the event stream. The token never enters the formatter, which is stronger than hoping a later filter removes it.

This ASCII and control-character fixture does not validate arbitrary UTF-8 or replace a mature JSON library. Production records also require size limits, stable schemas, and timestamp semantics.

Expected behavior or output

```
{"event": "rejected", "request": "job\u000a7", "reason": "syntax"}
```

Concrete experiment

Use quotes and backslashes in the label and parse the emitted line with a JSON parser. It must remain one event and must not contain the token.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

19.2 Platform Boundary

19.2.1 Engineering model

Centralized Logging — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

19.2.2 Production verification

Centralized Logging verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Separate stdout log collection from writable application storage

Keep stdout logging separate from the application's explicitly writable workspace.

example.py

```
import pathlib,yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
p=docs[0]['spec'];c=p['containers'][0]
assert c['env'][0]=={'name':'LOG_DESTINATION','value':'stdout'}
assert c['securityContext']['readOnlyRootFilesystem'] is True
assert p['volumes'][0]['emptyDir']['sizeLimit']=='32Mi'
print('offline log destination stdout; bounded scratch volume separate')
```

example.yaml

```
apiVersion: v1
kind: Pod
metadata: {name: harbor-logs}
spec:
  containers:
    - name: app
      image: harbor-service:lab
      imagePullPolicy: Never
      env: [{name: LOG_DESTINATION, value: stdout}]
      securityContext: {readOnlyRootFilesystem: true}
      volumeMounts: [{name: scratch, mountPath: /tmp}]
  volumes:
    - name: scratch
      emptyDir: {sizeLimit: 32Mi}
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

The application emits stdout instead of growing a log file inside its image filesystem. A cluster collector can consume the container stream independently of business-data volumes.

The Pod does not install a collector or configure rotation and retention. Those remain platform responsibilities, and stdout is not by itself a durable audit store.

Expected behavior or output

```
offline log destination stdout; bounded scratch volume separate
```

Concrete experiment

Simulate a collector outage and document local container-log limits. Repeated error logging must not fill the node filesystem.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-LOGGING. See the added references and revision index.

19.3 Failure and Recovery

19.3.1 Engineering model

Centralized Logging — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

19.3.2 Production verification

Centralized Logging verification — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Detect a gap in sequenced diagnostic records

Detect missing source-local log sequence numbers instead of assuming centralized evidence is complete.

example.py

```
records=[{'seq':1,'event':'start'},{'seq':2,'event':'accepted'},{'seq':5,'event':'done'}]
gaps=[]
for a,b in zip(records,records[1:]):
    if b['seq']!=a['seq']+1:gaps.append((a['seq']+1,b['seq']-1))
assert gaps==[(3,4)]
print('missing diagnostic sequence3..4; event completeness not established')
```

Run or inspect

```
python3 example.py
```

Explanation

The gap identifies missing sequence numbers three and four within one source identity. It does not reconstruct their contents or prove that the associated business operation failed.

Sampling, dropped events, restarts, and export failures all affect completeness. Carry process and build identity with sequence numbers so a restart is not mistaken for globally out-of-order history.

Expected behavior or output

```
missing diagnostic sequence3..4; event completeness not established
```

Concrete experiment

Add a new process identity starting at sequence one and reset tracking for that identity. Do not merge independent counters into one history.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-LOGGING. See the added references and revision index.

Worked example — Apply centralized logging to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
```

```
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 20 — SLIs, SLOs, and Error Budgets

This chapter develops **slis, slos, and error budgets** as a production mechanism whose application and platform responsibilities can be tested and recovered.

20.1 Defining the Contract

20.1.1 Engineering model

SLIs, SLOs, and Error Budgets — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop

admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

20.1.2 Production verification

SLIs, SLOs, and Error Budgets verification — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Define the SLI population before counting errors

Define an availability population that excludes health probes before calculating its success rate.

main.cpp

```
#include <array>
#include <iostream>

int main() {
    struct Request{bool user,successful;};const std::array
    trace{Request{true,true},Request{false,false},Request{true,false},Request{true,true}};
    unsigned eligible=0,good=0;for(auto r:trace)if(r.user){++eligible;good+=r.successful;}
    std::cout<<"good="<<good<<" eligible="<<eligible<<" health-probes-excluded\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The failed health probe does not enter the user-request denominator. The selected population has three requests and two successes, which answers a different question from all-HTTP-transaction success.

Eligibility must be specified before measurement, not used later to exclude inconvenient failures. Invalid requests and client cancellations also need explicit policies.

Expected behavior or output

```
good=2 eligible=3 health-probes-excluded
```

Concrete experiment

Add an invalid user request and decide eligibility from the published SLI contract. Keep that choice in the test rather than an ad hoc telemetry filter.

Validation of this new example: gcc: passed

Technical references: SRE-SLO. See the added references and revision index.

20.2 Implementing the Mechanism

20.2.1 Engineering model

SLIs, SLOs, and Error Budgets — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

20.2.2 Production verification

SLIs, SLOs, and Error Budgets verification — Implementing the Mechanism

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Express an availability burn-rate alert as a Prometheus rule

Express a burn-rate alert by normalizing errors against the allowed failure fraction.

example.py

```
import pathlib,yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
r=docs[0]['groups'][0]['rules'][0]
assert '/ 0.001 > 10' in r['expr']
assert r['for']=='5m' and r['labels']['severity']=='page'
print('offline alert rule: error fraction / 0.001 exceeds10 for5m')
```

example.yaml

```
groups:
  - name: harbor-slo
    rules:
      - alert: HarborFastBudgetBurn
        expr: |
          (sum(rate(harbor_requests_total{outcome="error"}[5m]))
           / clamp_min(sum(rate(harbor_requests_total[5m])), 1e-9)) / 0.001 > 10
        for: 5m
        labels: {severity: page}
        annotations:
          summary: "Harbor error budget is burning faster than10x"
```

Run or inspect

```
python3 example.py
# With Prometheus tooling installed:
promtool check rules example.yaml
```

Explanation

A 99.9% objective permits a failure fraction of 0.001. Dividing observed error fraction by 0.001 gives a burn ratio; ten means consuming allowance at ten times its steady rate.

The for clause requires persistence before firing. This one-window teaching rule is not a full multi-window production policy and can be noisy at low traffic; volume gates and longer windows need design.

Expected behavior or output

```
offline alert rule: error fraction / 0.001 exceeds10 for5m
```

Concrete experiment

Validate with promtool on the target system and add synthetic counter-series tests. YAML assertions cannot prove PromQL semantics or alert quality.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: PROM-ALERT, SRE-SLO. See the added references and revision index.

20.3 Verifying Production Behavior

20.3.1 Engineering model

SLIs, SLOs, and Error Budgets — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

20.3.2 Production verification

SLIs, SLOs, and Error Budgets verification — Verifying Production Behavior

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Calculate budget consumption without unsigned underflow

Clamp remaining error budget at zero and report excess failures separately.

main.cpp

```
#include <cstdint>
#include <iostream>

int main() {
    std::uint64_t total=100000,allowed=total/1000,failed=120;
    auto remaining=failed<=allowed?allowed-failed:0;
    auto excess=failed>allowed?failed-allowed:0;
    std::cout<<"allowed="<<allowed<<" remaining="<<remaining<<" excess="<<excess<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The fixture allows one hundred failures. Observing 120 exhausts the allowance and exceeds it by twenty; unchecked unsigned subtraction would create a falsely large positive remainder.

Exhaustion informs operational decisions rather than automatically prohibiting every change. Define which risk-reducing changes remain allowed under the error-budget policy.

Expected behavior or output

```
allowed=100 remaining=0 excess=20
```

Concrete experiment

Set failed to eighty and expect remaining twenty and excess zero. Keep the same population and rounding policy when comparing windows.

Validation of this new example: gcc: passed

Technical references: SRE-SLO. See the added references and revision index.

Worked example — Apply slis, slos, and error budgets to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
```

```
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 21 — Alerting and On-Call Engineering

This chapter develops **alerting and on-call engineering** as a production mechanism whose application and platform responsibilities can be tested and recovered.

21.1 Application Boundary

21.1.1 Engineering model

Alerting and On-Call Engineering — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation

must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

21.1.2 Production verification

Alerting and On-Call Engineering verification — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Page on a persistent user symptom, not every CPU spike

Require a persistent user-facing symptom before moving an alert into paging state.

main.cpp

```
#include <array>
#include <iostream>

int main() {
    unsigned consecutive=0,pages=0;bool firing=false;
    for(bool symptom:std::array{false,true,false,true,true,true}){
        consecutive=symptom?consecutive+1:0;
        bool next=consecutive>=3;if(next&&!firing)++pages;firing=next;
    }
    std::cout<<"page-transitions="<<pages<<" firing="<<std::boolalpha<<firing<<'\\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The first isolated failure clears before three consecutive samples accumulate. The final failure run triggers one paging transition rather than a new page for every sample.

The symptom should represent an actionable outcome such as failed user requests. High CPU can aid diagnosis but is not automatically an incident.

Expected behavior or output

```
page-transitions=1 firing=true
```

Concrete experiment

Add two healthy samples after the firing run and define recovery hysteresis. Avoid repeated transitions around one threshold.

Validation of this new example: gcc: passed

Technical references: SRE-ALERTING. See the added references and revision index.

21.2 Platform Boundary

21.2.1 Engineering model

Alerting and On-Call Engineering — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

21.2.2 Production verification

Alerting and On-Call Engineering verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Route actionable pages separately from warnings

Route pages separately from warnings with bounded Alertmanager grouping.

example.py

```
import pathlib,yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
c=docs[0]
assert c['route']['group_by']==['alertname','service']
assert c['route']['routes'][0]['receiver']=='oncall'
assert {r['name'] for r in c['receivers']}=={'warnings','oncall'}
print('offline routing: severity=page -> oncall; delivery integrations not configured')
```

example.yaml

```
route:
  receiver: warnings
  group_by: [alertname, service]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  routes:
    - matchers: ['severity="page"']
      receiver: oncall
receivers:
  - name: warnings
  - name: oncall
```

Run or inspect

```
python3 example.py
# With Alertmanager tooling installed:
amtool check-config example.yaml
```

Explanation

Grouping by alertname and service can reduce notification storms without using unbounded request IDs. The child page route selects oncall while other alerts use the default warnings receiver.

The receivers contain no delivery credentials and send nothing in this offline example. End-to-end notification still needs an authorized destination and a delivery test.

Expected behavior or output

```
offline routing: severity=page -> oncall; delivery integrations not configured
```

Concrete experiment

Use a mock private webhook to verify one grouped page for related alerts. Do not use real paging credentials without authorization.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: ALERTMANAGER. See the added references and revision index.

21.3 Failure and Recovery

21.3.1 Engineering model

Alerting and On-Call Engineering — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

21.3.2 Production verification

Alerting and On-Call Engineering verification — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Require a runbook action and owner for every page rule

Require an owner and response path for every paging rule.

example.py

```
rules=[{'name':'HighErrorRate','severity':'page','owner':'service-oncall','runbook':'runbooks/
errors.md'},
        {'name':'DiskTrend','severity':'warning','owner':'capacity-team'}]
def valid(r):return r['severity']!='page' or bool(r.get('owner') and r.get('runbook'))
assert all(map(valid,rules))
broken={'name':'Mystery','severity':'page'}
assert not valid(broken)
print('pages require owner and runbook; ownerless page rejected')
```

Run or inspect

```
python3 example.py
```

Explanation

The check enforces operational metadata, not whether the runbook is correct. It prevents an urgent page from being configured without an identified team or procedure.

Runbooks need bounded reversible actions and evidence checks. Automatically executing a dangerous command from an alert is a separate authorization decision, not part of this lint.

Expected behavior or output

```
pages require owner and runbook; ownerless page rejected
```

Concrete experiment

Check that the referenced runbook exists in version control and contains a recovery-verification step, not merely a restart command.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: SRE-ALERTING. See the added references and revision index.

Worked example — Apply alerting and on-call engineering to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
```

```
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 22 — CI for Production C++

This chapter develops **ci for production c++** as a production mechanism whose application and platform responsibilities can be tested and recovered.

22.1 Defining the Contract

22.1.1 Engineering model

CI for Production C++ — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

22.1.2 Production verification

CI for Production C++ verification — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally

complete.

NEW SECTION EXAMPLE

Define a CI gate that tests before producing a release artifact

Make release staging conditional on both compilation and application contract tests.

main.cpp

```
#include <array>
int square(int x){return x*x;}
int main(){for(auto
p:std::array{std::array{0,0},std::array{7,49},std::array{12,144}})if(square(p[0])!=p[1])return
1;}
```

ci.sh

```
set -eu
mkdir -p build stage
g++ -std=c++20 -Wall -Wextra -O2 main.cpp -o build/test
./build/test
cp build/test stage/verified-test
printf 'tests passed before artifact staging\n'
```

Run or inspect

```
sh ci.sh
```

Explanation

set -e and the executable's status prevent the copy step after a test failure. Compilation and successful execution are separate gates.

The small arithmetic fixture demonstrates CI ordering rather than a general overflow-safe API. Real release artifacts should also retain source and toolchain identity.

Expected behavior or output

```
tests passed before artifact staging
```

Concrete experiment

Change square to $x+x$ and verify that no new artifact is staged in a clean workspace. Do not mistake an old staging file for the result of the failed build.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: GCC-WARN. See the added references and revision index.

22.2 Implementing the Mechanism

22.2.1 Engineering model

CI for Production C++ — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

22.2.2 Production verification

CI for Production C++ verification — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Describe a native compiler matrix without hiding excluded configurations

Declare independent GCC and Clang build-and-test jobs in a compiler matrix.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
c=docs[0]; job=c['jobs']['build']
assert set(job['strategy']['matrix']['compiler'])=={'g++', 'clang++'}
assert job['strategy']['fail-fast'] is False
assert any('ctest' in s.get('run', '') for s in job['steps'])
print('offline workflow: GCC and Clang build/test jobs; no remote CI execution')
```

example.yaml

```
name: CppChecks
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        compiler: [g++, clang++]
    steps:
      - uses: actions/checkout@v4
      - name: Configure
        run: cmake -S . -B build -DCMAKE_CXX_COMPILER='${{ matrix.compiler }}'
      -DCMAKE_BUILD_TYPE=Debug
      - name: Build
        run: cmake --build build --parallel 2
      - name: Test
        run: ctest --test-dir build --output-on-failure
```

Run or inspect

```
python3 example.py
# Install this as .github/workflows/cpp.yml in an authorized repository to run CI.
```

Explanation

Each matrix job configures, builds, and tests its own tree. fail-fast false retains both outcomes instead of cancelling the sibling after the first failure.

The workflow assumes the compilers exist on the runner. ubuntu-latest and checkout@v4 are convenient mutable teaching references, not production pins. PyYAML can parse on as a YAML-1.1 boolean key; the workflow file retains GitHub's expected syntax.

Expected behavior or output

```
offline workflow: GCC and Clang build/test jobs; no remote CI execution
```

Concrete experiment

Add Release as another matrix dimension and keep output trees distinct. Linux GCC and Clang success does not establish MSVC coverage.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: GITHUB-MATRIX. See the added references and revision index.

22.3 Verifying Production Behavior

22.3.1 Engineering model

CI for Production C++ — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

22.3.2 Production verification

CI for Production C++ verification — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Reject promotion when a required test configuration is missing

Use an explicit required matrix instead of declaring success from whichever jobs happened to run.

example.py

```
required={'gcc-debug','gcc-release','clang-asan'}
results={'gcc-debug':'passed','gcc-release':'passed'}
def promote(r):return all(r.get(k)=='passed' for k in required)
assert not promote(results)
results['clang-asan']='passed';assert promote(results)
print('incomplete matrix blocks promotion; complete passing matrix permits review')
```

Run or inspect

```
python3 example.py
```

Explanation

Missing is not passed. The gate requires both presence and passing status for every required configuration before permitting release review.

The input dictionary is a fixture, not newly executed compiler evidence. A production gate must consume authenticated results bound to the same source and artifact identities.

Expected behavior or output

```
incomplete matrix blocks promotion; complete passing matrix permits review
```

Concrete experiment

Set one result to skipped and verify that promotion still blocks. Use an explicit approved exception rather than silently converting a skip into a pass.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: GITHUB-MATRIX. See the added references and revision index.

Worked example — Apply ci for production c++ to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }
```

```
int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 23 — Continuous Delivery and Deployment

This chapter develops **continuous delivery and deployment** as a production mechanism whose application and platform responsibilities can be tested and recovered.

23.1 Application Boundary

23.1.1 Engineering model

Continuous Delivery and Deployment — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation

must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

23.1.2 Production verification

Continuous Delivery and Deployment verification — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Check stored-schema compatibility before starting a new application build

A rolling deployment can run old and new binaries together. Make the readable schema interval an explicit application contract.

main.cpp

```
#include <array>
#include <iostream>

int main() {
    struct Build{int minimum_schema;int maximum_schema;};
    auto reads=[](Build b,int schema){return
b.minimum_schema<=schema&&schema<=b.maximum_schema;};
    Build old{1,2},next{2,3};
    for(int schema:std::array{1,2,3})
        std::cout<<"schema="<<schema<<" coexist="<<std::boolalpha
        <<(reads(old,schema)&&reads(next,schema))<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

reads compares a persisted schema version with the binary's supported interval. Schema 2 is the only version both builds understand, so it is the compatible intermediate state during this rollout.

This check addresses decoding compatibility, not the semantic correctness of a migration. Deploy an expand-compatible schema first, migrate consumers, and only then remove fields that the old binary still requires.

Expected behavior or output

```
schema=1 coexist=false
schema=2 coexist=true
schema=3 coexist=false
```

Concrete experiment

Change the next build's minimum schema to 3. The absence of overlap now blocks a mixed-version rollout; design a bridge release rather than disabling the check.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

23.2 Platform Boundary

23.2.1 Engineering model

Continuous Delivery and Deployment — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

23.2.2 Production verification

Continuous Delivery and Deployment verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Configure a rolling Deployment with an explicit availability budget

The controller may temporarily create one extra Pod but should not deliberately reduce the available replica count during this rollout.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
s=docs[0]['spec'];r=s['strategy']['rollingUpdate']
assert r['maxSurge']==1 and r['maxUnavailable']==0
assert s['progressDeadlineSeconds']>s['minReadySeconds']
print('offline rollout: desired=3; surge=1; unavailable=0; stable-ready=10s')
```

example.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: harbor-rollout}
spec:
  replicas: 3
  minReadySeconds: 10
  progressDeadlineSeconds: 180
  revisionHistoryLimit: 3
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
  selector:
    matchLabels: {app: harbor-rollout}
  template:
    metadata:
      labels: {app: harbor-rollout}
    spec:
      containers:
        - name: app
          image: harbor-service:lab-v2
          imagePullPolicy: Never
          ports: [{name: http, containerPort: 8080}]
          readinessProbe:
            httpGet: {path: /readyz, port: http}
            periodSeconds: 2
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

maxSurge permits an additional Pod while the old replicas remain. maxUnavailable zero constrains the controller's rollout actions; it cannot prevent unrelated node failures or guarantee that readiness correctly represents useful service.

minReadySeconds requires sustained readiness before a new Pod counts as available. progressDeadlineSeconds reports a stalled rollout rather than automatically proving that rolling back application data would be safe.

Expected behavior or output

```
offline rollout: desired=3; surge=1; unavailable=0; stable-ready=10s
```

Concrete experiment

On an isolated cluster, make the new build's readiness endpoint fail. Observe the rollout stall while old replicas remain; inspect events and conditions before deciding whether to undo it.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-DEPLOYMENT, K8S-PROBES. See the added references and revision index.

23.3 Failure and Recovery

23.3.1 Engineering model

Continuous Delivery and Deployment — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

23.3.2 Production verification

Continuous Delivery and Deployment verification — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume

unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Require client evidence as well as controller completion

A rollout can be mechanically complete while the new version returns incorrect application results.

example.py

```
def accept_release(status, probes):
    controller_ok=(status['observedGeneration']==status['generation']
                    and status['updated']==status['desired']
                    and status['available']==status['desired'])
    application_ok=bool(probes) and all(p['status']==200 and p['result']==49 for p in probes)
    return controller_ok and application_ok
state=dict(generation=8,observedGeneration=8,updated=3,available=3,desired=3)
assert not accept_release(state,[{'status':200,'result':48}])
assert accept_release(state,[{'status':200,'result':49}])
print('controller complete + incorrect result: reject; correct result: accept review')
```

Run or inspect

```
python3 example.py
```

Explanation

The generation comparison prevents interpreting a previous controller observation as evidence about the new desired state. Replica counts then check that the rollout has reached its declared shape.

The synthetic client probe checks an application answer, not merely an HTTP status. In a deployment pipeline, bind both observations to the same artifact and configuration; this fixture supplies evidence-checking logic, not a live rollout result.

Expected behavior or output

```
controller complete + incorrect result: reject; correct result: accept review
```

Concrete experiment

Leave all replica counts green but lower observedGeneration to 7. The release should still be rejected because the controller has not observed generation 8.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-DEPLOYMENT. See the added references and revision index.

Worked example — Apply continuous delivery and deployment to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 24 — Canary and Blue-Green Releases

This chapter develops **canary and blue-green releases** as a production mechanism whose application and platform responsibilities can be tested and recovered.

24.1 Defining the Contract

24.1.1 Engineering model

Canary and Blue-Green Releases — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume

unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

24.1.2 Production verification

Canary and Blue-Green Releases verification — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Compare error rates rather than raw canary error counts

Canary and baseline receive different traffic volumes, so a promotion gate must compare rates over declared populations.

example.py

```
def assess(base,canary):
    if min(base['requests'],canary['requests'])<1000:return 'insufficient'
    b=base['errors']/base['requests'];c=canary['errors']/canary['requests']
    return 'hold' if c>b+0.005 else 'candidate'
assert assess({'requests':10000,'errors':20},{'requests':1000,'errors':12})=='hold'
assert assess({'requests':10000,'errors':20},{'requests':100,'errors':0})=='insufficient'
print('canary 1.2% vs baseline 0.2%: hold; 100 requests: insufficient')
```

Run or inspect

```
python3 example.py
```

Explanation

Twelve errors are fewer than twenty, but the canary's denominator is much smaller: 12/1000 is 1.2%, compared with 20/10000 at 0.2%. The illustrative absolute-difference gate therefore holds promotion.

The minimum sample count avoids interpreting a tiny perfect sample as convincing evidence. This is a transparent teaching policy, not a statistical significance test; real cohorts also need comparable request types, observation windows, and an uncertainty analysis.

Expected behavior or output

```
canary 1.2% vs baseline 0.2%: hold; 100 requests: insufficient
```

Concrete experiment

Supply a canary with 1000 requests and 2 errors. It becomes a candidate under this policy, but still needs latency and functional checks before promotion.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: SRE-CANARY. See the added references and revision index.

24.2 Implementing the Mechanism

24.2.1 Engineering model

Canary and Blue-Green Releases — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

24.2.2 Production verification

Canary and Blue-Green Releases verification — Implementing the Mechanism

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Represent a blue-green cutover as a Service selector change

Keep both application versions addressable by labels and switch only the Service's selected release.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
s=docs[0]['spec'];pods=[{'app': 'harbor-front', 'release': 'blue'},
                        {'app': 'harbor-front', 'release': 'green'}]
selected=[p for p in pods if all(p.get(k)==v for k,v in s['selector'].items())]
assert selected==[pods[1]]
print('offline selector chooses green; blue remains outside the Service')
```

example.yaml

```
apiVersion: v1
kind: Service
metadata: {name: harbor-front}
spec:
  selector: {app: harbor-front, release: green}
  ports: [{name: http, port: 80, targetPort: 8080}]
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

The selector requires both app and release labels, so the Service routes to green endpoints instead of matching every version with the same application name. The Python fixture demonstrates label matching only.

A real cutover still requires ready green Pods, endpoint propagation, compatible data, and connection-draining policy. Changing a selector does not instantly migrate existing TCP connections or undo writes made by the new application.

Expected behavior or output

```
offline selector chooses green; blue remains outside the Service
```

Concrete experiment

Change release to blue in the manifest and rerun the label check. On a lab cluster, verify both newly established requests and existing long-lived connections separately.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-SERVICE. See the added references and revision index.

24.3 Verifying Production Behavior

24.3.1 Engineering model

Canary and Blue-Green Releases — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

24.3.2 Production verification

Canary and Blue-Green Releases verification — Verifying Production

Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Reject a canary whose fast average hides an unacceptable tail

Use a declared nearest-rank percentile to detect a small set of very slow requests.

example.py

```
import math
samples=[10]*95+[200]*5
ordered=sorted(samples)
p99=ordered[math.ceil(.99*len(ordered))-1]
mean=sum(ordered)/len(ordered)
assert mean<25 and p99>100
print(f'mean={mean:.1f}ms p99={p99}ms: hold release')
```

Run or inspect

```
python3 example.py
```

Explanation

The index $\text{ceil}(0.99 \cdot N) - 1$ selects the nearest-rank 99th percentile from sorted observations. Five slow requests leave the mean at 19.5 ms but make p99 equal to 200 ms.

The 100 ms limit is an illustrative release threshold tied to this fixture. Retain the measurement population and method; a percentile without sample count, units, or workload scope cannot justify a rollout decision.

Expected behavior or output

```
mean=19.5ms p99=200ms: hold release
```

Concrete experiment

Change the five slow requests from 200 ms to 80 ms and recompute both statistics. Explain why a tail-specific gate can change even when the typical request is identical.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: SRE-CANARY. See the added references and revision index.

Worked example — Apply canary and blue-green releases to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
```

```
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 25 — GitOps and Declarative Operations

This chapter develops **gitops and declarative operations** as a production mechanism whose application and platform responsibilities can be tested and recovered.

25.1 Application Boundary

25.1.1 Engineering model

GitOps and Declarative Operations — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation

must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

25.1.2 Production verification

GitOps and Declarative Operations verification — Application Boundary

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Compare desired application configuration with observed configuration

Keep GitOps reconciliation focused on declared fields rather than replacing every field returned by a platform API.

example.py

```
desired={'workers':4,'queue_limit':64,'log_level':'info'}
observed={'workers':4,'queue_limit':256,'log_level':'info','uptime_seconds':900}
drift={k:(observed.get(k),v) for k,v in desired.items() if observed.get(k)!=v}
assert drift=={'queue_limit':(256,64)}
print('queue_limit: observed=256 desired=64; uptime is not managed configuration')
```

Run or inspect

```
python3 example.py
```

Explanation

The comprehension iterates over desired keys, which defines this reconciler's ownership boundary. It reports the accidental queue increase but does not attempt to reset uptime, a runtime observation.

A production controller needs field ownership, authorization, conflict handling, and version checks. This small diff is the application-side contract that prevents read-only status from being mistaken for desired configuration.

Expected behavior or output

```
queue_limit: observed=256 desired=64; uptime is not managed configuration
```

Concrete experiment

Add a desired log_level of debug and confirm that only those two managed fields are proposed for change. Do not treat arbitrary platform defaults as drift.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-KUSTOMIZE. See the added references and revision index.

25.2 Platform Boundary

25.2.1 Engineering model

GitOps and Declarative Operations — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

25.2.2 Production verification

GitOps and Declarative Operations verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Render a Kustomize overlay without modifying its base

Build a production-shaped overlay from a shared Deployment and a small declarative replica patch.

base/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: harbor-gitops}
spec:
  replicas: 1
  selector: {matchLabels: {app: harbor-gitops}}
  template:
    metadata: {labels: {app: harbor-gitops}}
    spec:
      containers:
        - {name: app, image: 'harbor-service:lab', imagePullPolicy: Never}
```

base/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources: [deployment.yaml]
```

overlays/production/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources: [../../base]
replicas:
  - name: harbor-gitops
    count: 3
```

check.py

```
from pathlib import Path
import yaml
b=yaml.safe_load(Path('base/deployment.yaml').read_text())
o=yaml.safe_load(Path('overlays/production/kustomization.yaml').read_text())
assert b['spec']['replicas']==1
assert o['replicas']==[{ 'name': 'harbor-gitops', 'count': 3}]
print('offline base=1, overlay=3; Kustomize rendering requires kubectl')
```

Run or inspect

```
python3 check.py
kubectl kustomize overlays/production
```

Explanation

The base records the reusable object. The overlay names that base and changes only the replica count, so environment-specific configuration does not require copying the entire Deployment.

The included Python check verifies the declared inputs. `kubectl kustomize` performs the actual composition on a machine with that tool; inspect its rendered YAML before any cluster apply. Neither rendering nor this offline check proves runtime readiness.

Expected behavior or output

```
offline base=1, overlay=3; Kustomize rendering requires kubectl
```

Concrete experiment

Create a staging overlay with count 2 and render both overlays. Verify that editing staging does not alter the production output or the base file.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-KUSTOMIZE. See the added references and revision index.

25.3 Failure and Recovery

25.3.1 Engineering model

GitOps and Declarative Operations — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

25.3.2 Production verification

GitOps and Declarative Operations verification — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume

unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Revert an emergency configuration change with an auditable Git commit

Use a private repository to demonstrate how a temporary live fix becomes a reviewed version-control decision.

lab.py

```
import pathlib, subprocess, tempfile
with tempfile.TemporaryDirectory() as td:
    p = pathlib.Path(td)
    def git(*args): return
    subprocess.run(['git', '-C', td, *args], check=True, capture_output=True, text=True).stdout.strip()
    git('init', '-q'); git('config', 'user.name', 'Lab'); git('config', 'user.email', 'lab@example.invalid')
    (p/'service.conf').write_text('workers=4\n'); git('add', 'service.conf'); git('commit', '-qm', 'baseline')
    (p/'service.conf').write_text('workers=8\n'); git('commit', '-qam', 'temporary emergency capacity')
    git('revert', '--no-edit', 'HEAD')
    assert (p/'service.conf').read_text() == 'workers=4\n'
    assert len(git('rev-list', 'HEAD').splitlines()) == 3
    print('baseline restored; emergency change and its revert remain in history')
```

Run or inspect

```
python3 lab.py
```

Explanation

git revert adds a new inverse commit instead of erasing the emergency commit. That preserves the evidence that capacity was temporarily raised and later restored.

In GitOps, changing only the live cluster can be undone by reconciliation. Capture the intended fix in the source of desired state, or use a documented emergency pause and reconciliation procedure; this lab changes only its temporary local repository.

Expected behavior or output

```
baseline restored; emergency change and its revert remain in history
```

Concrete experiment

Inspect git log in a retained copy of the lab repository. Compare reverting a commit with resetting history, and explain why an auditable inverse is preferable on a shared deployment branch.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: GIT-REVERT. See the added references and revision index.

Worked example — Apply gitops and declarative operations to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 26 — Infrastructure as Code

This chapter develops **infrastructure as code** as a production mechanism whose application and platform responsibilities can be tested and recovered.

26.1 Defining the Contract

26.1.1 Engineering model

Infrastructure as Code — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

26.1.2 Production verification

Infrastructure as Code verification — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally

complete.

NEW SECTION EXAMPLE

Reject an infrastructure request that violates an application sizing contract

Terraform variable validation can keep a workload's minimum replica requirement beside the infrastructure declaration.

main.tf

```
terraform { required_version = ">= 1.4.0" }
variable "replicas" {
  type    = number
  default = 3
  validation {
    condition     = var.replicas >= 2 && floor(var.replicas) == var.replicas
    error_message = "replicas must be an integer of at least two."
  }
}
resource "terraform_data" "service_contract" {
  input = { replicas = var.replicas, port = 8080 }
}
output "contract" { value = terraform_data.service_contract.output }
```

Run or inspect

```
terraform init
terraform plan -var=replicas=3
terraform plan -var=replicas=1
```

Explanation

The validation rejects both one replica and fractional values before the contract can be used. A numeric Terraform variable does not itself imply an integer, so floor enforces that additional requirement.

terraform_data records local managed state without provisioning a cloud resource. This is a safe way to study plan and state semantics; the constraint still represents an application requirement rather than evidence of actual availability.

Expected behavior or output

```
terraform plan -var=replicas=1 reports the custom validation error.
terraform plan -var=replicas=3 proposes one terraform_data resource.
```

Concrete experiment

Run terraform init and both plans in a disposable directory. Do not run apply against an unrelated workspace; this example does not need cloud credentials.

Validation of this new example: gcc: target required

Technical references: TERRAFORM-DATA, TERRAFORM-VALIDATE. See the added references and revision index.

26.2 Implementing the Mechanism

26.2.1 Engineering model

Infrastructure as Code — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

26.2.2 Production verification

Infrastructure as Code verification — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Review a Terraform plan for replacement before approving it

Resource replacement often contains both delete and create actions, so a gate must inspect the whole action list.

example.py

```
plan={'resource_changes':[
    {'address':'terraform_data.service','change':{'actions':['update']}},
    {'address':'terraform_data.volume_contract','change':{'actions':['delete','create']}}}
destructive=[r['address'] for r in plan['resource_changes']
              if 'delete' in r['change']['actions']]
assert destructive==['terraform_data.volume_contract']
print('manual review required: terraform_data.volume_contract')
```

Run or inspect

```
python3 example.py
```

Explanation

The membership test finds deletion even when creation follows it. Looking only at the last action would misclassify replacement as harmless resource creation.

This input is a deliberately supplied plan-shaped fixture. For a real review, obtain JSON from `terraform show -json` on the saved plan, protect its potentially sensitive values, and apply the same reviewed plan rather than regenerating an unreviewed one.

Expected behavior or output

```
manual review required: terraform_data.volume_contract
```

Concrete experiment

Change the actions to create then delete and confirm it still requires review. Lifecycle ordering changes availability details, not the fact that an existing resource will be destroyed.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: TERRAFORM-JSON. See the added references and revision index.

26.3 Verifying Production Behavior

26.3.1 Engineering model

Infrastructure as Code — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown,

file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

26.3.2 Production verification

Infrastructure as Code verification — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Detect changed infrastructure inputs between review and execution

A plan approval should not silently authorize a different set of application sizing inputs.

example.py

```
import hashlib, json
approved={'replicas':3,'memory_mib':128,'revision':'release-a'}
def fingerprint(x):return
hashlib.sha256(json.dumps(x,sort_keys=True,separators=(',',':')).encode()).hexdigest()
recorded=fingerprint(approved)
actual=dict(approved);actual['memory_mib']=64
assert fingerprint(actual)!=recorded
assert fingerprint(dict(reversed(list(approved.items()))))==recorded
print('key order does not matter; changed memory request invalidates review')
```

Run or inspect

```
python3 example.py
```

Explanation

Canonical JSON serialization removes incidental dictionary order and spacing from the fingerprint. The changed memory value produces a different identity, so a previous approval cannot be reused unchanged.

A checksum detects a mismatch only when the approved checksum itself is trusted. It does not authenticate a plan, secure a CI runner, or replace signed provenance; those controls belong around the review record.

Expected behavior or output

key order does not matter; changed memory request invalidates review

Concrete experiment

Alter the revision while retaining the old replica count. Require a new review because application identity is part of the infrastructure contract too.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: CPP20. See the added references and revision index.

Worked example — Apply infrastructure as code to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```

#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}

```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal

communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 27 — Cloud Architecture for C++ Services

This chapter develops **cloud architecture for c++ services** as a production mechanism whose application and platform responsibilities can be tested and recovered.

27.1 Application Boundary

27.1.1 Engineering model

Cloud Architecture for C++ Services — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation

must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

27.1.2 Production verification

Cloud Architecture for C++ Services verification — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Express cloud dependencies without embedding provider credentials

The application needs a storage operation, not a permanent cloud access key in its domain model.

main.cpp

```
#include <map>
#include <memory>
#include <string>
#include <string_view>
#include <iostream>

int main() {
    struct ObjectStore{virtual ~ObjectStore()=default;virtual void put(std::string
key,std::string value)=0;};
    struct MemoryStore final:ObjectStore{std::map<std::string,std::string> objects;void
put(std::string k,std::string v)override{objects[std::move(k)]=std::move(v);}};
    auto archive=[](ObjectStore&store,std::string_view
id){store.put(std::string(id),"completed");};
    MemoryStore fake;archive(fake,"job-7");
    std::cout<<"stored="<<fake.objects.at("job-7")<<"; domain API contains no credential
parameter\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

ObjectStore gives archive a narrow operation whose contract can be tested with MemoryStore. The domain operation neither selects a cloud provider nor receives a secret string.

A real cloud adapter must acquire authorized short-lived credentials through the selected provider's supported identity mechanism and enforce scope. The in-memory implementation tests the application boundary only; it does not implement or validate cloud IAM.

Expected behavior or output

```
stored=completed; domain API contains no credential parameter
```

Concrete experiment

Make the fake store reject writes and return a typed error from the interface. Verify that archive reports a storage failure without logging credentials or silently claiming completion.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

27.2 Platform Boundary

27.2.1 Engineering model

Cloud Architecture for C++ Services — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

27.2.2 Production verification

Cloud Architecture for C++ Services verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally

complete.

NEW SECTION EXAMPLE

Declare zone spread while preserving unschedulable evidence

A Deployment can request that replicas be spread across labeled zones, but it cannot invent capacity in a missing zone.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
p=docs[0]['spec']['template'];c=p['spec']['topologySpreadConstraints'][0]
assert c['labelSelector']['matchLabels']==p['metadata']['labels']
assert c['whenUnsatisfiable']=='DoNotSchedule' and c['maxSkew']==1
print('offline zone-spread constraint matches the workload labels')
```

example.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: harbor-zones}
spec:
  replicas: 3
  selector: {matchLabels: {app: harbor-zones}}
  template:
    metadata: {labels: {app: harbor-zones}}
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: DoNotSchedule
          labelSelector: {matchLabels: {app: harbor-zones}}
      containers:
        - name: app
          image: harbor-service:lab
          imagePullPolicy: Never
          resources:
            requests: {cpu: 100m, memory: 64Mi}
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

The topology key names the node label that defines failure domains. The matching selector ensures the scheduler counts this workload's Pods when evaluating skew.

DoNotSchedule makes the constraint a scheduling requirement rather than a preference. It does not guarantee three distinct zones: actual eligible node labels, domains, capacity, and scheduler rules determine placement. Inspect Pending events instead of assuming three replicas imply zone resilience.

Expected behavior or output

```
offline zone-spread constraint matches the workload labels
```

Concrete experiment

On a labeled test cluster, remove capacity in one eligible domain and inspect scheduling decisions. Compare that behavior with `ScheduleAnyway` before choosing the availability policy.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-TOPOLOGY. See the added references and revision index.

27.3 Failure and Recovery

27.3.1 Engineering model

Cloud Architecture for C++ Services — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

27.3.2 Production verification

Cloud Architecture for C++ Services verification — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume

unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Calculate surviving capacity after one zone is lost

Availability planning must evaluate the degraded topology, not only total healthy capacity.

example.py

```
capacity={'zone-a':400,'zone-b':400,'zone-c':400}
demand=700
remaining={lost:sum(v for k,v in capacity.items() if k!=lost) for lost in capacity}
assert min(remaining.values())==800
headroom=min(remaining.values())-demand
print(f'worst surviving capacity=800 requests/s; headroom={headroom} requests/s')
assert demand<=min(remaining.values())
```

Run or inspect

```
python3 example.py
```

Explanation

The comprehension evaluates each single-zone failure and takes the worst surviving capacity. With identical zones, two remaining zones supply 800 requests per second against assumed demand of 700.

The inputs are planning assumptions, not measured cloud capacity. Real load balancing, dependencies, connection rebuilding, and recovery traffic can reduce usable throughput; qualify these numbers with a failure-state load test.

Expected behavior or output

```
worst surviving capacity=800 requests/s; headroom=100 requests/s
```

Concrete experiment

Raise demand to 900 and observe the failed capacity assertion. Adding a nominal third replica is not enough when the remaining zones cannot absorb its load.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: CPP20. See the added references and revision index.

Worked example — Apply cloud architecture for c++ services to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 28 — Failure Injection and Chaos Engineering

This chapter develops **failure injection and chaos engineering** as a production mechanism whose application and platform responsibilities can be tested and recovered.

28.1 Defining the Contract

28.1.1 Engineering model

Failure Injection and Chaos Engineering — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation

must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

28.1.2 Production verification

Failure Injection and Chaos Engineering verification — Defining the Contract

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Define a fault experiment with an explicit abort condition

A chaos experiment should specify the injected fault, observed request population, and maximum allowed impact before it starts.

example.py

```
experiment={'target':'lab-worker','fault':'delay_ms','value':200,
           'duration_s':30,'max_error_rate':0.02,'max_targets':1}
assert experiment['duration_s']<=60 and experiment['max_targets']==1
observations=[(1000,5),(1000,25)]
for requests,errors in observations:
    abort=errors/requests>experiment['max_error_rate']
    print(f'errors={errors}/{requests} abort={str(abort).lower()}')
```

Run or inspect

```
python3 example.py
```

Explanation

The experiment record constrains both duration and target count, while `max_error_rate` defines the condition that ends the exercise. The second observation exceeds the two-percent bound and triggers abort.

An abort decision is useful only when the injection mechanism can actually be stopped and the service can recover. This fixture evaluates the policy without delaying any real workload; an authorized lab must test the stop path before broader experiments.

Expected behavior or output

```
errors=5/1000 abort=false
errors=25/1000 abort=true
```

Concrete experiment

Add a zero-request observation and represent it as insufficient evidence rather than dividing by zero or declaring the service healthy.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: CPP20. See the added references and revision index.

28.2 Implementing the Mechanism

28.2.1 Engineering model

Failure Injection and Chaos Engineering — Implementing the Mechanism

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

28.2.2 Production verification

Failure Injection and Chaos Engineering verification — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Inject a deterministic slow dependency without sleeping in tests

Use an explicit elapsed-time result to exercise deadline behavior without a flaky wall-clock delay.

main.cpp

```
#include <chrono>
#include <optional>
#include <iostream>

int main() {
    using namespace std::chrono_literals;
    struct Reply{int value;std::chrono::milliseconds elapsed;};
    auto call=[](bool slow){return Reply{49,slow?200ms:20ms};};
    auto request=[&](bool slow)->std::optional<int>{auto
r=call(slow);if(r.elapsed>100ms)return {};return r.value;};
    auto fast=request(false),slow=request(true);
    std::cout<<"fast="<<*fast<<"  slow-accepted="<<std::boolalpha<<slow.has_value()<<"\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The fake dependency reports elapsed time as data, allowing the test to exercise both sides of the 100 ms deadline deterministically. The slow result is not admitted into the successful response path.

This checks the application's response to a missed budget, not actual cancellation of in-flight I/O. A real client must use a transport deadline and define what happens when the remote operation completes after the caller stops waiting.

Expected behavior or output

```
fast=49 slow-accepted=false
```

Concrete experiment

Set elapsed time to exactly 100 ms and choose whether the deadline is inclusive. Encode that decision in a boundary test rather than letting a comparison operator decide accidentally.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

28.3 Verifying Production Behavior

28.3.1 Engineering model

Failure Injection and Chaos Engineering — Verifying Production Behavior

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

28.3.2 Production verification

Failure Injection and Chaos Engineering verification — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Prove fault removal before declaring a recovery experiment complete

Passing a client probe while the injection flag remains enabled is not a completed recovery.

example.py

```
def recovered(state):
    return (not state['fault_enabled'] and state['successes']==state['probes']
            and state['probes']>=20 and state['queue_depth']==0)
s={'fault_enabled':True,'successes':20,'probes':20,'queue_depth':0}
assert not recovered(s)
s['fault_enabled']=False;assert recovered(s)
s['queue_depth']=4;assert not recovered(s)
print('recovery requires fault removed, 20/20 probes, and drained queue')
```

Run or inspect

```
python3 example.py
```

Explanation

The gate checks restoration of the test environment, functional recovery, and backlog drainage as separate facts. A momentary successful request does not establish that accumulated work has cleared.

The twenty-probe threshold is a lab policy, not proof of long-term reliability. Record the observation window and continue normal monitoring after the gate; this fixture demonstrates what a recovery report must contain.

Expected behavior or output

```
recovery requires fault removed, 20/20 probes, and drained queue
```

Concrete experiment

Leave one probe failed and verify the experiment cannot be reported as cleanly recovered. Preserve the failed observation for the postmortem.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: CPP20. See the added references and revision index.

Worked example — Apply failure injection and chaos engineering to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
```

```

#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}

```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 29 — Incident Response and Postmortems

This chapter develops **incident response and postmortems** as a production mechanism whose application and platform responsibilities can be tested and recovered.

29.1 Application Boundary

29.1.1 Engineering model

Incident Response and Postmortems — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation

must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

29.1.2 Production verification

Incident Response and Postmortems verification — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Emit an incident event without exposing request secrets

Application events should carry operation identity and failure category while excluding credentials and raw payloads.

main.cpp

```
#include <string>
#include <string_view>
#include <iostream>

int main() {
    struct Request{std::string id,authorization,payload;};
    Request r{"req-17","Bearer SECRET","private customer data"};
    auto incident_event=[](const Request&x){return "request="+x.id+"
category=dependency_timeout"}};
    const auto event=incident_event(r);

    if(event.find("SECRET")!=std::string::npos||event.find("private")!=std::string::npos)return 1;
    std::cout<<event<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

`incident_event` explicitly selects the request identifier and the timeout category. It does not serialize the whole request, which would include an authorization token and private content.

The test checks that the known sensitive values do not appear in this output. Real identifiers also need size and encoding limits; redaction is easier when secret-bearing fields never enter the logging API.

Expected behavior or output

```
request=req-17 category=dependency_timeout
```

Concrete experiment

Add a deployment identifier as a separate trusted field. Do not add the raw Authorization header merely to help correlate the incident.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

29.2 Platform Boundary

29.2.1 Engineering model

Incident Response and Postmortems — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

29.2.2 Production verification

Incident Response and Postmortems verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Order platform and application evidence in a UTC incident timeline

Normalize offsets before correlating a deployment event with application errors.

example.py

```
from datetime import datetime, timezone
rows=[('2026-09-01T12:00:05Z', 'errors increased'),
      ('2026-09-01T08:00:00-04:00', 'release applied'),
      ('2026-09-01T12:01:00+00:00', 'rollback requested')]
parsed=[(datetime.fromisoformat(t.replace('Z', '+00:00')).astimezone(timezone.utc), e) for t, e
in rows]
for t, e in sorted(parsed): print(t.strftime('%H:%M:%SZ'), e)
```

Run or inspect

```
python3 example.py
```

Explanation

The release timestamp uses a minus-four-hour offset, so 08:00 local corresponds to 12:00 UTC. Sorting parsed instants yields the actual chronological order instead of sorting incompatible timestamp strings.

Temporal proximity supports investigation but does not prove the release caused the error increase. Preserve event source and clock uncertainty when combining evidence from hosts with imperfect synchronization.

Expected behavior or output

```
12:00:00Z release applied
12:00:05Z errors increased
12:01:00Z rollback requested
```

Concrete experiment

Add a timestamp without an offset and reject it before correlation. Guessing a timezone during an incident can produce a convincing but false timeline.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: CPP20. See the added references and revision index.

29.3 Failure and Recovery

29.3.1 Engineering model

Incident Response and Postmortems — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

29.3.2 Production verification

Incident Response and Postmortems verification — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown,

file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Turn a postmortem finding into a testable corrective action

A corrective action needs an owner, a reproducible failure, and acceptance evidence rather than a vague instruction to improve monitoring.

example.py

```

action={'id':'INC-17-A1','owner':'service-team',
        'failure_fixture':'dependency_slow_200ms',
        'acceptance':'returns deadline_exceeded within configured transport deadline',
        'test':'deadline_regression','status':'open'}
required=('owner','failure_fixture','acceptance','test')
assert all(action.get(k) for k in required)
assert action['status']!='closed'
results={'deadline_regression':'passed'}
if results.get(action['test'])=='passed':action['status']='ready_for_review'
print('INC-17-A1: ready_for_review, not automatically closed')

```

Run or inspect

```
python3 example.py
```

Explanation

The action ties the incident's slow-dependency condition to a named regression test and a measurable outcome. Owner and acceptance fields make responsibility and completion review explicit.

A passing fixture advances the action to review rather than closing it automatically. Deployment, monitoring, and verification on the intended environment may still be required; the dictionary is a workflow demonstration, not a real incident record.

Expected behavior or output

```
INC-17-A1: ready_for_review, not automatically closed
```

Concrete experiment

Remove the owner and ensure the action fails validation. An unowned task is not a dependable prevention mechanism.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: CPP20. See the added references and revision index.

Worked example — Apply incident response and postmortems to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```

#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

```

```
int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 30 — Capacity and Performance Engineering

This chapter develops **capacity and performance engineering** as a production mechanism whose application and platform responsibilities can be tested and recovered.

30.1 Defining the Contract

30.1.1 Engineering model

Capacity and Performance Engineering — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation

must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

30.1.2 Production verification

Capacity and Performance Engineering verification — Defining the Contract

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Estimate average concurrency from arrival rate and residence time

Apply Little's-law arithmetic to a stable illustrative workload before choosing queue and worker budgets.

example.py

```
arrival_per_s=200
mean_residence_s=0.05
mean_in_system=arrival_per_s*mean_residence_s
print(f'assumed stable workload: mean in-system={mean_in_system:.1f} requests')
assert mean_in_system==10
```

Run or inspect

```
python3 example.py
```

Explanation

The product of 200 requests per second and 0.05 seconds gives an average of ten requests in the system. The units cancel to requests, which helps catch a milliseconds-versus-seconds error.

This is an average relationship under a stable workload, not a maximum queue length or a promise about p99 latency. Burst size, service-time variance, downstream limits, and failure conditions still determine the required headroom.

Expected behavior or output

```
assumed stable workload: mean in-system=10.0 requests
```

Concrete experiment

Increase mean residence time to 0.2 seconds while holding arrivals constant. Explain why average in-flight work rises to forty even though demand did not increase.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: CPP20. See the added references and revision index.

30.2 Implementing the Mechanism

30.2.1 Engineering model

Capacity and Performance Engineering — Implementing the Mechanism

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

30.2.2 Production verification

Capacity and Performance Engineering verification — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Load a bounded service queue and count deliberate rejections

A capacity experiment should account for offered, accepted, completed, and rejected work independently.

main.cpp

```
#include <deque>
#include <iostream>

int main() {
    std::deque<int> queue; unsigned offered=0, accepted=0, rejected=0, completed=0;
    for(int id=0; id<10; ++id) { ++offered; if(queue.size()==3) ++rejected; else { queue.push_back(id);
    ++accepted; }}
    while(!queue.empty()) { queue.pop_front(); ++completed; }
    if(offered!=accepted+rejected || accepted!=completed) return 1;
    std::cout<<"offered="<<offered<<" accepted="<<accepted<<" rejected="<<rejected<<"
    completed="<<completed<<"'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Ten requests arrive before any service step. The three-slot queue admits exactly three and rejects seven rather than expanding until memory runs out.

The accounting equations ensure the experiment does not silently lose offered work. This deterministic burst model tests admission policy, not concurrent throughput; a live load generator must also record achieved arrival rate and latency for rejected requests.

Expected behavior or output

```
offered=10 accepted=3 rejected=7 completed=3
```

Concrete experiment

Interleave one completion after every two arrivals and compare accepted counts. The arrival pattern, not just total request count, changes queue pressure.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

30.3 Verifying Production Behavior

30.3.1 Engineering model

Capacity and Performance Engineering — Verifying Production Behavior

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

30.3.2 Production verification

Capacity and Performance Engineering verification — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown,

file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Check capacity headroom against both normal and degraded measurements

A deployment sized only for healthy average throughput can fail its planned single-instance outage.

example.py

```

measured={'healthy_capacity_rps':1200,'one_instance_down_rps':750}
target_peak_rps=700
reserve_fraction=0.10
required=target_peak_rps/(1-reserve_fraction)
normal=measured['healthy_capacity_rps']>=required
degraded=measured['one_instance_down_rps']>=required
print(f'required={required:.1f}rps normal={normal} degraded={degraded}')
assert normal and not degraded

```

Run or inspect

```
python3 example.py
```

Explanation

To retain ten percent spare capacity at a 700-request peak, the system needs about 777.8 requests per second of tested capacity. The healthy configuration passes, while the degraded fixture at 750 does not.

The values are explicit example measurements, not observations from a cloud deployment. In a real review, retain the load-test configuration, latency target, duration, and dependency behavior that define each capacity number.

Expected behavior or output

```
required=777.8rps normal=True degraded=False
```

Concrete experiment

Lower peak demand to 650 and recalculate the gate. Keep the degraded-capacity test in the release plan rather than assuming autoscaling can instantly replace a failed instance.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: CPP20. See the added references and revision index.

Worked example — Apply capacity and performance engineering to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```

#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {

```



```

std::signal(SIGTERM, request_stop);
std::signal(SIGINT, request_stop);
std::cout << "ready=1\n";
while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
std::cout << "ready=0 draining=1\n";
std::cout << "shutdown=complete\n";
}

```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 31 — Cloud Cost Engineering

This chapter develops **cloud cost engineering** as a production mechanism whose application and platform responsibilities can be tested and recovered.

31.1 Application Boundary

31.1.1 Engineering model

Cloud Cost Engineering — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop

admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

31.1.2 Production verification

Cloud Cost Engineering verification — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Attribute assumed service cost to successfully completed requests

Use a declared accounting period and denominator rather than quoting a bare cloud bill as application efficiency.

example.py

```
compute=120.0;storage=30.0;egress=10.0 # invented monthly dollars for this exercise
completed=2_000_000
cost_per_million=(compute+storage+egress)/completed*1_000_000
print(f'illustrative cost per million successful requests=${cost_per_million:.2f}')
assert cost_per_million==80
```

Run or inspect

```
python3 example.py
```

Explanation

The numerator includes all three stated cost categories for the same period. Dividing by completed requests, rather than all attempts, avoids hiding wasted retries in a misleadingly large denominator.

These are invented accounting inputs, not provider prices or the user's finances. A real allocation should include shared infrastructure rules, failed-work costs, discounts, and the exact period; it is not automatically a profitability measure.

Expected behavior or output

```
illustrative cost per million successful requests=$80.00
```

Concrete experiment

Halve completed requests while costs stay fixed. The resulting \$160 per million reveals that lower utilization can raise unit cost without any price change.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: CPP20. See the added references and revision index.

31.2 Platform Boundary

31.2.1 Engineering model

Cloud Cost Engineering — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

31.2.2 Production verification

Cloud Cost Engineering verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Reduce resource requests without removing a tested replica floor

Cost changes should be represented as reviewable resource settings, not an undocumented reduction in redundancy.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
s=docs[0]['spec'];r=s['template']['spec']['containers'][0]['resources']
assert s['replicas']==3 and r['requests']['cpu']=='150m'
assert int(r['requests']['memory'][:-2])<int(r['limits']['memory'][:-2])
print('offline plan preserves three replicas and declares 150m/96Mi requests')
```

example.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: harbor-cost-lab}
spec:
  replicas: 3
  selector: {matchLabels: {app: harbor-cost-lab}}
  template:
    metadata: {labels: {app: harbor-cost-lab}}
    spec:
      containers:
      - name: app
        image: harbor-service:lab
        imagePullPolicy: Never
        resources:
          requests: {cpu: 150m, memory: 96Mi}
          limits: {memory: 192Mi}
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

The request values influence scheduling and any utilization calculations that use requests as a denominator. The memory limit is a separate runtime ceiling; it is not a reservation of that amount.

This manifest proposes settings but supplies no evidence that 96 MiB is sufficient. Validate memory high-water marks, restart behavior, and degraded capacity before accepting a cost reduction. Leaving the CPU limit unspecified does not grant unlimited node capacity.

Expected behavior or output

```
offline plan preserves three replicas and declares 150m/96Mi requests
```

Concrete experiment

Compare autoscaling decisions before and after reducing the CPU request while the actual CPU use remains unchanged. The utilization percentage can rise solely because its denominator changed.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-RESOURCES, K8S-HPA. See the added references and revision index.

31.3 Failure and Recovery

31.3.1 Engineering model

Cloud Cost Engineering — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy,

degraded, and failed states from available evidence, the design is not operationally complete.

31.3.2 Production verification

Cloud Cost Engineering verification — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Reject a cheaper configuration that violates a latency requirement

Cost optimization is constrained by service requirements rather than choosing the cheapest row unconditionally.

example.py

```

candidates=[{'name':'small','monthly':90,'p99_ms':180},
             {'name':'medium','monthly':140,'p99_ms':85},
             {'name':'large','monthly':220,'p99_ms':60}]
qualified=[c for c in candidates if c['p99_ms']<=100]
choice=min(qualified,key=lambda c:c['monthly'])
assert choice['name']=='medium'
print('illustrative qualified choice=medium; cheapest small fails p99<=100ms')

```

Run or inspect

```
python3 example.py
```

Explanation

The latency filter is applied before minimizing cost, so the small configuration is not eligible despite its lower assumed price. The medium configuration is the cheapest among those satisfying this fixture's requirement.

The table is a synthetic planning example, not a benchmark or current price list. A real decision must also constrain reliability, memory, concurrency, and recovery performance, and retain the evidence for each candidate.

Expected behavior or output

```
illustrative qualified choice=medium; cheapest small fails p99<=100ms
```

Concrete experiment

Add a minimum surviving-zone throughput constraint and see whether the same candidate remains eligible. Do not optimize one scalar cost while discarding the service contract.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: CPP20. See the added references and revision index.

Worked example — Apply cloud cost engineering to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```

#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}

```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 32 — Backup and Disaster Recovery

This chapter develops **backup and disaster recovery** as a production mechanism whose application and platform responsibilities can be tested and recovered.

32.1 Defining the Contract

32.1.1 Engineering model

Backup and Disaster Recovery — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop

admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

32.1.2 Production verification

Backup and Disaster Recovery verification — Defining the Contract should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Take a database-aware SQLite backup rather than copying a live file

Use SQLite's backup interface so the application state is copied through the database engine.

example.py

```
import pathlib,sqlite3,tempfile
with tempfile.TemporaryDirectory() as td:
    root=pathlib.Path(td)
    with sqlite3.connect(root/'live.db') as live:
        live.execute('CREATE TABLE jobs(id TEXT PRIMARY KEY, result INTEGER NOT NULL)')
        live.execute('INSERT INTO jobs VALUES(?,?)',('a',49));live.commit()
    with sqlite3.connect(root/'backup.db') as backup:
        live.backup(backup)
        assert backup.execute('SELECT result FROM jobs WHERE id=?',('a',)).fetchone()==(49,)
        live.execute('UPDATE jobs SET result=64 WHERE id=?',('a',));live.commit()
    with sqlite3.connect(root/'backup.db') as restored:
        assert restored.execute('SELECT result FROM jobs').fetchone()==(49,)
    print('backup preserves committed result=49 independently of later live update=64')
```

Run or inspect

```
python3 example.py
```

Explanation

live.backup asks SQLite to copy a consistent database through its supported API. It does not simply copy live.db while ignoring a WAL or concurrent database state.

The later update distinguishes the backup from the live connection: reopening the backup still returns 49. This local test establishes logical backup behavior, not protection against losing the host or proof of a production recovery-time objective.

Expected behavior or output

```
backup preserves committed result=49 independently of later live update=64
```

Concrete experiment

Add another committed job before backup and verify both rows after restore. Store production backups in a separate failure domain with access control and retention.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: [SQLITE-BACKUP](#). See the added references and revision index.

32.2 Implementing the Mechanism

32.2.1 Engineering model

Backup and Disaster Recovery — Implementing the Mechanism should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

32.2.2 Production verification

Backup and Disaster Recovery verification — Implementing the Mechanism

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

Schedule a non-overlapping backup Job with bounded execution

A CronJob needs a concurrency policy and failure limits in addition to a schedule expression.

example.py

```
import pathlib, yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
s=docs[0]['spec'];j=s['jobTemplate']['spec']
assert s['concurrencyPolicy']=='Forbid' and s['timeZone']=='Etc/UTC'
assert j['activeDeadlineSeconds']==600 and j['backoffLimit']==1
print('offline backup schedule: UTC 02:00; no overlap; 600s job deadline')
```

example.yaml

```
apiVersion: batch/v1
kind: CronJob
metadata: {name: harbor-backup}
spec:
  schedule: "0 2 * * *"
  timeZone: Etc/UTC
  concurrencyPolicy: Forbid
  startingDeadlineSeconds: 300
  successfulJobsHistoryLimit: 2
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 1
      activeDeadlineSeconds: 600
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: backup
              image: harbor-backup:lab
              imagePullPolicy: Never
              command: ["/usr/local/bin/backup-sqlite"]
              args: ["/data/live.db", "/backup"]
              volumeMounts:
                - {name: source, mountPath: /data, readOnly: true}
                - {name: backups, mountPath: /backup}
          volumes:
            - name: source
              persistentVolumeClaim: {claimName: harbor-data}
            - name: backups
              persistentVolumeClaim: {claimName: harbor-backups}
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

Forbid prevents overlapping Jobs from this CronJob; activeDeadlineSeconds bounds a job's runtime and backoffLimit limits retries. Neither setting guarantees exactly one successful backup per day.

The image, executable, and PVCs are explicit lab prerequisites, not supplied cluster resources. A read-only mounted live SQLite database may need a compatible WAL access arrangement; qualify the backup image and storage access mode with the

running database before deploying this schedule.

Expected behavior or output

offline backup schedule: UTC 02:00; no overlap; 600s job deadline

Concrete experiment

Simulate a backup that lasts beyond the next scheduled interval. Record the skipped run and alert on backup freshness rather than treating CronJob existence as proof of recovery capability.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-CRONJOB, SQLITE-BACKUP. See the added references and revision index.

32.3 Verifying Production Behavior

32.3.1 Engineering model

Backup and Disaster Recovery — Verifying Production Behavior should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

32.3.2 Production verification

Backup and Disaster Recovery verification — Verifying Production Behavior

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact

inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Verify restored business state and report modeled recovery objectives

An intact file is not enough: a restore must reopen and answer an application-level query.

example.py

```
import sqlite3, pathlib, tempfile
with tempfile.TemporaryDirectory() as td:
    p=pathlib.Path(td)/'restored.db'
    with sqlite3.connect(p) as db:
        db.execute('CREATE TABLE jobs(id TEXT PRIMARY KEY, state TEXT, result INTEGER)')
        db.execute("INSERT INTO jobs VALUES('job-7','done',49)");db.commit()
    with sqlite3.connect(p) as db:
        assert db.execute('PRAGMA integrity_check').fetchone()=='ok',)
        assert db.execute("SELECT state,result FROM jobs WHERE id='job-7'").fetchone()=='done',49)
        backup_time=1000;incident_time=1300;recovery_start=1305;service_verified=1340
        print(f'restore query=done/49; modeled data age={incident_time-backup_time}s;
        recovery={service_verified-recovery_start}s')
```

Run or inspect

```
python3 example.py
```

Explanation

PRAGMA integrity_check examines database structure, while the SELECT verifies an expected business record. Both are needed because structural integrity alone does not prove the right backup was restored.

The timestamps are explicitly modeled values, not measured recovery performance. The 300-second backup age bounds potential exposure in this scenario, while the 35-second recovery interval ends at verified service behavior rather than at file-copy completion.

Expected behavior or output

```
restore query=done/49; modeled data age=300s; recovery=35s
```

Concrete experiment

Replace the restored result with 48 while leaving the database structurally valid. The business assertion should catch this wrong-state restore.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: SQLITE-C. See the added references and revision index.

Worked example — Apply backup and disaster recovery to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Chapter 33 — Building the HarborDispatch Production Platform

This chapter develops **building the harbordispatch production platform** as a production mechanism whose application and platform responsibilities can be tested and recovered.

33.1 Application Boundary

33.1.1 Engineering model

Building the HarborDispatch Production Platform — Application Boundary

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation

must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

33.1.2 Production verification

Building the HarborDispatch Production Platform verification — Application Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Keep HarborDispatch job decisions independent of deployment state

The production-platform capstone needs a small application core whose admission and result rules remain testable outside Kubernetes.

main.cpp

```
#include <map>
#include <optional>
#include <string>
#include <iostream>

int main() {
    struct Job{int input;std::optional<int> result;};
    std::map<std::string,Job> jobs;
    auto submit=[&](std::string id,int input){
        if(input<0||input>100)return std::string("invalid");
        if(auto i=jobs.find(id);i!=jobs.end())return
std::string(i->second.input==input?"replay":"conflict");
        if(jobs.size()==2)return std::string("busy");
        jobs.emplace(std::move(id),Job{input,{}});return std::string("accepted");};
    std::cout<<submit("j1",7)<<' '<<submit("j1",7)<<' '<<submit("j1",8)<<'\n';
    for(auto&[id,j]:jobs)if(!j.result)j.result=j.input*j.input;
    std::cout<<"result="<<*jobs.at("j1").result<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The operation ID distinguishes a safe duplicate from a conflicting request that reuses the same identity with different input. Result calculation is independent of Pod names, replica count, and routing.

The map is volatile and capacity counts retained jobs, so this is a bounded core demonstration rather than durable distributed dispatch. A real deployment must define retention, transactional persistence, and cross-replica deduplication before advertising those guarantees.

Expected behavior or output

```
accepted replay conflict
result=49
```

Concrete experiment

Submit two different IDs and then a third. Test busy independently of replay, and specify when completed records may be evicted without losing retry protection.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

33.2 Platform Boundary

33.2.1 Engineering model

Building the HarborDispatch Production Platform — Platform Boundary

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

33.2.2 Production verification

Building the HarborDispatch Production Platform verification — Platform Boundary should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally

complete.

NEW SECTION EXAMPLE

Assemble the platform-facing Deployment and Service contract

Wire the lab HTTP service to named ports, probes, non-root execution, and explicit resource requests.

example.py

```
import pathlib,yaml
docs=list(yaml.safe_load_all(pathlib.Path("example.yaml").read_text()))
d,s=docs;p=d['spec']['template'];c=p['spec']['containers'][0]
assert s['spec']['selector']==p['metadata']['labels']
assert s['spec']['ports'][0]['targetPort']==c['ports'][0]['name']
assert c['readinessProbe']['httpGet']['path']=='/readyz'
print('offline integration: selector, named port, probes, resources, and identity agree')
```

example.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: harbor-dispatch-lab}
spec:
  replicas: 2
  selector: {matchLabels: {app: harbor-dispatch-lab}}
  template:
    metadata: {labels: {app: harbor-dispatch-lab}}
    spec:
      automountServiceAccountToken: false
      terminationGracePeriodSeconds: 10
      securityContext: {runAsUser: 10001, runAsNonRoot: true}
      containers:
        - name: app
          image: harbor-dispatch:lab
          imagePullPolicy: Never
          args: ["--container"]
          ports: [{name: http, containerPort: 8080}]
          readinessProbe:
            httpGet: {path: /readyz, port: http}
            periodSeconds: 2
          livenessProbe:
            httpGet: {path: /healthz, port: http}
            periodSeconds: 5
          resources:
            requests: {cpu: 100m, memory: 32Mi}
            limits: {memory: 64Mi}
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: {drop: [ALL]}
---
apiVersion: v1
kind: Service
metadata: {name: harbor-dispatch-lab}
spec:
  selector: {app: harbor-dispatch-lab}
  ports: [{name: http, port: 80, targetPort: http}]
```

Run or inspect

```
python3 example.py
# On a disposable configured Kubernetes cluster only:
kubectl apply --dry-run=server -f example.yaml
```

Explanation

The Service selector matches the Deployment template labels, and targetPort resolves to the container's named http port. The probes call different paths so readiness and process health can evolve independently.

The image is a lab prerequisite and must contain the service from section 33.3 with its container entrypoint. The provided server is deliberately tiny and unauthenticated; do not expose this teaching endpoint publicly or confuse offline manifest agreement with live-cluster qualification.

Expected behavior or output

offline integration: selector, named port, probes, resources, and identity agree

Concrete experiment

Change targetPort to an absent name and make the offline integration assertion fail. Then test actual endpoint routing on a private cluster before deployment.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: K8S-DEPLOYMENT, K8S-SERVICE, K8S-SECURITY. See the added references and revision index.

33.3 Failure and Recovery

33.3.1 Engineering model

Building the HarborDispatch Production Platform — Failure and Recovery

should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so

scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

33.3.2 Production verification

Building the HarborDispatch Production Platform verification — Failure and Recovery should be treated as an operating contract, not a product checkbox. A production C++ service must preserve application correctness while infrastructure restarts processes, changes routing, rotates configuration, constrains resources, and experiences partial failures. Begin by naming what the application guarantees and what the platform is allowed to change.

Trace the complete path from request admission to completion. Record process and network boundaries, configuration sources, resource ownership, telemetry, and failure responses. Native services must make signal handling, thread shutdown, file-descriptor ownership, connection draining, allocation behavior, and deadlines explicit. Infrastructure automation cannot repair an application with an undefined lifecycle.

Bound every capacity: queues, workers, connections, retries, timeouts, log buffers, and memory. When demand exceeds sustained capacity, choose an explicit response such as backpressure, rejection, degradation, or controlled shedding. Adding replicas can increase capacity while also multiplying pressure on downstream systems, so scaling decisions require system-level measurements.

Verification must match the claim. A successful deployment command proves that an API accepted desired state; it does not prove clients receive correct responses. Combine platform state with application-level requests, telemetry, and failure tests. Verify network and identity changes from outside the workload.

Production evidence should include source revision, compiler and build configuration, artifact digest, deployment configuration, environment identity, tests, and observation window. This makes rollback refer to an exact artifact and lets incident analysis distinguish code, configuration, and infrastructure changes.

Automation increases consistency, not correctness. Declarative controllers continuously reconcile toward the state they were given, even when that state is wrong. Use layered evidence: static checks, unit and integration tests, artifact inspection, pre-production validation, progressive exposure, service-level observation, and a tested rollback path.

Assume dependencies become slow rather than simply unavailable. Timeouts must bound waiting, retries must not amplify traffic without limit, and queue saturation must be observable. Assume telemetry export fails; monitoring must not consume unbounded application memory. Assume termination arrives during active work; stop admission, drain for a bounded interval, and then exit predictably.

Before accepting the mechanism, name its owner, measurable success condition, rollback trigger, and recovery procedure. If operators cannot distinguish healthy, degraded, and failed states from available evidence, the design is not operationally complete.

NEW SECTION EXAMPLE

Run a real local HTTP readiness, computation, and shutdown check

This executable capstone boundary serves three GET endpoints and is exercised through actual loopback TCP requests. It is not a full HTTP implementation or a durable job service.

server.cpp

```

#include <arpa/inet.h>
#include <sys/socket.h>
#include <poll.h>
#include <unistd.h>
#include <csignal>
#include <charconv>
#include <cerrno>
#include <iostream>
#include <string>
#include <string_view>
#include <stdexcept>
struct FD { int value; explicit FD(int x):value(x){if(x<0)throw std::runtime_error("socket failure");}
~FD(){if(value>0)::close(value);} FD(const FD&)=delete;FD&operator=(const FD&)=delete;};
volatile std::sig_atomic_t stopping=0;
extern "C" void stop_handler(int){stopping=1;}
int main(int argc,char**argv){try{
std::signal(SIGTERM,stop_handler);std::signal(SIGINT,stop_handler);
const bool container=argc==2&&std::string_view(argv[1])=="--container";
FD listener(::socket(AF_INET,SOCK_STREAM,0));
sockaddr_in address{};address.sin_family=AF_INET;
address.sin_addr.s_addr=htonl(container?INADDR_ANY:INADDR_LOOPBACK);
address.sin_port=htons(container?8080:0);
if(!::bind(listener.value,reinterpret_cast<sockaddr*>(&address),sizeof
address)<0)||::listen(listener.value,8)<0)return 2;
socklen_t length=sizeof address;
if(!::getsockname(listener.value,reinterpret_cast<sockaddr*>(&address),&length)<0)return 3;
std::cout<<"READY ";<<ntohs(address.sin_port)<<'\n'<<std::flush;
while(!stopping){
pollfd ready{listener.value,POLLIN,0};int n=::poll(&ready,1,100);
if(n<0){if(errno==EINTR)continue;return 4;};if(n==0)continue;
int raw=::accept(listener.value,nullptr,nullptr);if(raw<0){if(errno==EINTR)continue;return
5;};FD client(raw);
timeval timeout{1,0};::setsockopt(client.value,SOL_SOCKET,SO_RCVTIME0,&timeout,sizeof
timeout);

::setsockopt(client.value,SOL_SOCKET,SO_SNDTIME0,&timeout,sizeof timeout);
std::string request;char bytes[512];
while(request.find("\r\n\r\n")==std::string::npos&&request.size()<4096){
auto got=::recv(client.value,bytes,sizeof bytes,0);if(got<=0)break;
request.append(bytes,static_cast<std::size_t>(got));
}
int status=400;std::string body="bad request\n";
auto end=request.find("\r\n");

if(request.size()<=4096&&end!=std::string::npos&&request.find("\r\n\r\n")!=std::string::npos){
std::string_view line(request.data(),end);
if(line.starts_with("GET ")&&line.ends_with(" HTTP/1.1")){
auto path=line.substr(4,line.size()-4-9);
if(path=="healthz"||path=="readyz"){status=200;body="ok\n";}
else if(path.starts_with("/square?x=")){
auto text=path.substr(10);int x{};auto
[p,ec]=std::from_chars(text.data(),text.data()+text.size(),x);
if(ec==std::errc{}&&p==text.data()+text.size()&&x==0&&x<=100){status=200;body=std::to_str
ing(x*x)+"\n";}
}else{status=404;body="not found\n";}
}
}
std::string response="HTTP/1.1 "+std::to_string(status)+(status==200?" OK":status==404?" Not
Found":" Bad Request")+
"\r\nContent-Type: text/plain\r\nConnection: close\r\nContent-Length:
"+std::to_string(body.size())+"\r\n\r\n"+body;
std::size_t sent=0;while(sent<response.size()){
auto count=::send(client.value,response.data()+sent,response.size()-sent,MSG_NOSIGNAL);

```



```

    if(count<0&&errno==EINTR)continue;if(count<=0)break;sent+=static_cast<std::size_t>(count);
}
}
std::cout<<"STOPPED\n";
}catch(const std::exception&e){std::cerr<<e.what()<<\n';return 9;}}

```

Dockerfile

```

FROM debian:bookworm-slim AS build
RUN apt-get update && apt-get install -y --no-install-recommends g++ && rm -rf
/var/lib/apt/lists/*
WORKDIR /src
COPY server.cpp .
RUN g++ -std=c++20 -O2 server.cpp -o server
FROM debian:bookworm-slim
COPY --from=build /src/server /usr/local/bin/harbor-dispatch
USER 10001
ENTRYPOINT ["/usr/local/bin/harbor-dispatch"]
CMD ["--container"]

```

lab.py

```

import http.client,selectors,subprocess
p=subprocess.Popen(['./server'],stdout=subprocess.PIPE,stderr=subprocess.PIPE,text=True)
try:
    selector=selectors.DefaultSelector();selector.register(p.stdout,selectors.EVENT_READ)
    if not selector.select(5):raise RuntimeError('server never became ready')
    line=p.stdout.readline().strip();assert line.startswith('READY ')
    port=int(line.split()[1]);selector.close()
    def get(path):
        c=http.client.HTTPConnection('127.0.0.1',port,timeout=3)
        try:
            c.request('GET',path);r=c.getresponse();return r.status,r.read().decode()
        finally:c.close()
    assert get('/readyz')==('200','ok\n')
    assert get('/healthz')==('200','ok\n')
    assert get('/square?x=7')==('200','49\n')
    assert get('/square?x=bad')[0]==400
    assert get('/missing')[0]==404
    p.terminate();out,err=p.communicate(timeout=3)
    assert p.returncode==0 and 'STOPPED' in out and not err
    print('loopback readiness, health, square=49, 400, 404, and SIGTERM exit verified')
finally:
    if p.poll() is None:p.kill();p.communicate()

```

run.sh

```

#!/bin/sh
set -eu
g++ -std=c++20 -O2 server.cpp -o server
python3 lab.py

```

Run or inspect

```

sh run.sh
# Optional private container lab:
docker build -t harbor-dispatch:lab .

```

Explanation

The listener binds an ephemeral loopback port for host tests and prints it only after bind and listen succeed. poll limits idle shutdown delay; the signal handler merely sets sig_atomic_t state, leaving cleanup to ordinary control flow and FD destructors.

Requests are bounded to 4096 bytes with a receive timeout, the integer query is fully parsed and range-checked, and responses include Content-Length plus Connection: close. The test makes real HTTP requests and checks bodies as well as status. This single-connection-at-a-time teaching parser omits TLS, authentication, general HTTP semantics, durable jobs, and production request scheduling.

Expected behavior or output

loopback readiness, health, square=49, 400, 404, and SIGTERM exit verified

Concrete experiment

Add a test for $x=100$ and $x=101$, then hold an incomplete connection open and observe the receive timeout. Treat Docker image building and Kubernetes deployment as separate tests; the passing loopback harness does not qualify them.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: LINUX-SOCKET, LINUX-POLL, LINUX-RECV, LINUX-SEND, DOCKER-MULTISTAGE. See the added references and revision index.

Worked example — Apply building the harbordispatch production platform to HarborDispatch

Requirement and design

Define one client-visible acceptance check, one platform-state check, and one rollback condition before applying the change. Keep artifact identity immutable and record the environment in which verification occurs.

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <iostream>
#include <thread>

std::atomic_bool stopping{false};
extern "C" void request_stop(int) { stopping.store(true); }

int main() {
    std::signal(SIGTERM, request_stop);
    std::signal(SIGINT, request_stop);
    std::cout << "ready=1\n";
    while (!stopping.load()) std::this_thread::sleep_for(std::chrono::milliseconds(50));
    std::cout << "ready=0 draining=1\n";
    std::cout << "shutdown=complete\n";
}
```

This program demonstrates a lifecycle boundary, not a complete service. Integrate termination with the actual listener, worker pool, request deadlines, and telemetry. More complex cleanup belongs in ordinary program flow after the signal communicates intent.

Evidence and failure clinic

Confirm readiness, request termination, and require bounded exit. Under load, readiness should stop new traffic before termination. If shutdown exceeds its grace period, locate the unbounded operation rather than merely increasing the timeout. If retries ignore cancellation, termination may never converge.

Practice and implementation lab

Wrap a small C++ HTTP or TCP service with this chapter's mechanism. Add success, malformed-input, dependency-failure, overload, and shutdown cases. Capture enough telemetry to explain each outcome.

Acceptance criteria. The service exposes meaningful readiness, stays within stated resource bounds, terminates within a defined interval, and connects platform changes to client-visible evidence.

Chapter takeaway. Production infrastructure is useful only when it makes application behavior more controlled, observable, and recoverable.

Appendix A — Production C++ Service Checklist

Record executable and artifact digest, compiler configuration, service identity, interfaces and ports, resource requests and limits, probe semantics, termination period, queue capacities, downstream timeouts, retry budgets, certificate sources, telemetry destinations, SLOs, alert ownership, rollback procedure, and recovery owner.

Appendix B — Kubernetes and Deployment Review

Inspect image identity, security context, resources, probes, service account, configuration and secret mounts, networking assumptions, disruption behavior, topology constraints, and rollout strategy. Generated manifests are not evidence that the rendered state is correct.

Appendix C — Observability Review

Metrics require units and bounded cardinality. Logs require structure, retention, and redaction. Traces require propagation and sampling policy. SLOs require a population, time window, and success definition. Alerts should identify conditions requiring action.

Appendix D — Incident and Recovery Exercise

Exercise a release that increases latency while retries amplify downstream load and one zone becomes unavailable. Identify the service-level symptom, freeze unrelated changes, reduce amplification, decide whether to roll back, preserve evidence, communicate status, and verify recovery from a client perspective.

Appendix E — Glossary

Artifact digest — Content-derived identity for an immutable artifact.

Backpressure — A bounded mechanism limiting upstream production.

Error budget — Unsuccessful service behavior permitted by an SLO.

Readiness — Whether a workload should receive new traffic.

SLI — A measured indicator of service behavior.

SLO — A target for an SLI over a defined population and window.

Closing Perspective

Cloud-native infrastructure does not replace systems programming. It makes lifecycle and failure boundaries more frequent and explicit. The operating discipline remains: define the contract, trace the mechanism, bound resources, verify independently, measure the complete path, and preserve recovery.

Section example revision index

The original book text and original code are retained. New examples are independently authored additions for the numbered sections listed below. Their validation records do not certify the retained original manuscript or every optional platform procedure.

1. 1.1 Application Boundary
2. 1.2 Platform Boundary
3. 1.3 Failure and Recovery
4. 2.1 Defining the Contract
5. 2.2 Implementing the Mechanism
6. 2.3 Verifying Production Behavior
7. 3.1 Application Boundary
8. 3.2 Platform Boundary
9. 3.3 Failure and Recovery
10. 4.1 Defining the Contract
11. 4.2 Implementing the Mechanism
12. 4.3 Verifying Production Behavior
13. 5.1 Application Boundary
14. 5.2 Platform Boundary
15. 5.3 Failure and Recovery
16. 6.1 Defining the Contract
17. 6.2 Implementing the Mechanism
18. 6.3 Verifying Production Behavior
19. 7.1 Application Boundary
20. 7.2 Platform Boundary
21. 7.3 Failure and Recovery
22. 8.1 Defining the Contract
23. 8.2 Implementing the Mechanism
24. 8.3 Verifying Production Behavior
25. 9.1 Application Boundary
26. 9.2 Platform Boundary
27. 9.3 Failure and Recovery

- 28. 10.1 Defining the Contract
- 29. 10.2 Implementing the Mechanism
- 30. 10.3 Verifying Production Behavior
- 31. 11.1 Application Boundary
- 32. 11.2 Platform Boundary
- 33. 11.3 Failure and Recovery
- 34. 12.1 Defining the Contract
- 35. 12.2 Implementing the Mechanism
- 36. 12.3 Verifying Production Behavior
- 37. 13.1 Application Boundary
- 38. 13.2 Platform Boundary
- 39. 13.3 Failure and Recovery
- 40. 14.1 Defining the Contract
- 41. 14.2 Implementing the Mechanism
- 42. 14.3 Verifying Production Behavior
- 43. 15.1 Application Boundary
- 44. 15.2 Platform Boundary
- 45. 15.3 Failure and Recovery
- 46. 16.1 Defining the Contract
- 47. 16.2 Implementing the Mechanism
- 48. 16.3 Verifying Production Behavior
- 49. 17.1 Application Boundary
- 50. 17.2 Platform Boundary
- 51. 17.3 Failure and Recovery
- 52. 18.1 Defining the Contract
- 53. 18.2 Implementing the Mechanism
- 54. 18.3 Verifying Production Behavior
- 55. 19.1 Application Boundary
- 56. 19.2 Platform Boundary
- 57. 19.3 Failure and Recovery
- 58. 20.1 Defining the Contract
- 59. 20.2 Implementing the Mechanism

- 60. 20.3 Verifying Production Behavior
- 61. 21.1 Application Boundary
- 62. 21.2 Platform Boundary
- 63. 21.3 Failure and Recovery
- 64. 22.1 Defining the Contract
- 65. 22.2 Implementing the Mechanism
- 66. 22.3 Verifying Production Behavior
- 67. 23.1 Application Boundary
- 68. 23.2 Platform Boundary
- 69. 23.3 Failure and Recovery
- 70. 24.1 Defining the Contract
- 71. 24.2 Implementing the Mechanism
- 72. 24.3 Verifying Production Behavior
- 73. 25.1 Application Boundary
- 74. 25.2 Platform Boundary
- 75. 25.3 Failure and Recovery
- 76. 26.1 Defining the Contract
- 77. 26.2 Implementing the Mechanism
- 78. 26.3 Verifying Production Behavior
- 79. 27.1 Application Boundary
- 80. 27.2 Platform Boundary
- 81. 27.3 Failure and Recovery
- 82. 28.1 Defining the Contract
- 83. 28.2 Implementing the Mechanism
- 84. 28.3 Verifying Production Behavior
- 85. 29.1 Application Boundary
- 86. 29.2 Platform Boundary
- 87. 29.3 Failure and Recovery
- 88. 30.1 Defining the Contract
- 89. 30.2 Implementing the Mechanism
- 90. 30.3 Verifying Production Behavior
- 91. 31.1 Application Boundary

- 92. 31.2 Platform Boundary
- 93. 31.3 Failure and Recovery
- 94. 32.1 Defining the Contract
- 95. 32.2 Implementing the Mechanism
- 96. 32.3 Verifying Production Behavior
- 97. 33.1 Application Boundary
- 98. 33.2 Platform Boundary
- 99. 33.3 Failure and Recovery

Technical references for the new examples

ALERTMANAGER — Prometheus: Alertmanager configuration

<https://prometheus.io/docs/alerting/latest/configuration/>

CPP20 — C++20 working draft N4861: language and standard-library clauses

<https://timsong-cpp.github.io/cppwp/n4861/>

DOCKER-CONTEXT — Docker: build context and dockerignore

<https://docs.docker.com/build/concepts/context/>

DOCKER-MULTISTAGE — Docker: multi-stage builds

<https://docs.docker.com/build/building/multi-stage/>

DOCKER-PUSH — Docker: image push

<https://docs.docker.com/reference/cli/docker/image/push/>

DOCKER-RUN — Docker: container run reference

<https://docs.docker.com/reference/cli/docker/container/run/>

GCC-WARN — GCC: warning options

<https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html>

GIT-REVERT — Git: revert

<https://git-scm.com/docs/git-revert>

GITHUB-MATRIX — GitHub Actions: matrix job strategies

<https://docs.github.com/en/actions/using-jobs/using-a-matrix-for-your-jobs>

ISTIO-MTLS — Istio: PeerAuthentication policy

https://istio.io/latest/docs/reference/config/security/peer_authentication/

ISTIO-TRAFFIC — Istio: traffic management concepts

<https://istio.io/latest/docs/concepts/traffic-management/>

K8S-CONFIG — Kubernetes: ConfigMaps

<https://kubernetes.io/docs/concepts/configuration/configmap/>

K8S-CRONJOB — Kubernetes: CronJobs

<https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/>

K8S-DEPLOYMENT — Kubernetes: Deployments

<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

K8S-DNS — Kubernetes: DNS for Services and Pods

<https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/>

K8S-ENDPOINTS — Kubernetes: EndpointSlices

<https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>

K8S-HPA — Kubernetes: horizontal Pod autoscaling

<https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>

K8S-JOB — Kubernetes: Jobs

<https://kubernetes.io/docs/concepts/workloads/controllers/job/>

K8S-KUSTOMIZE — Kubernetes: declarative management using Kustomize

<https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/>

K8S-LOGGING — Kubernetes: logging architecture

<https://kubernetes.io/docs/concepts/cluster-administration/logging/>

K8S-POD — Kubernetes: Pod lifecycle and identity

<https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>

K8S-PROBES — Kubernetes: startup, liveness, and readiness probes

<https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>

K8S-RESOURCES — Kubernetes: container resource requests and limits

<https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>

K8S-SECRET — Kubernetes: Secrets

<https://kubernetes.io/docs/concepts/configuration/secret/>

K8S-SECURITY — Kubernetes: configure a security context

<https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>

K8S-SERVICE — Kubernetes: Services

<https://kubernetes.io/docs/concepts/services-networking/service/>

K8S-STATEFUL — Kubernetes: StatefulSets

<https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/>

K8S-STORAGE — Kubernetes: persistent volumes

<https://kubernetes.io/docs/concepts/storage/persistent-volumes/>

K8S-TOPOLOGY — Kubernetes: topology spread constraints

<https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/>

LINUX-LOADER — Linux/POSIX manual: ld.so.8

<https://man7.org/linux/man-pages/man8/ld.so.8.html>

LINUX-POLL — Linux man-pages: poll

<https://man7.org/linux/man-pages/man2/poll.2.html>

LINUX-RECV — Linux/POSIX manual: recv.2

<https://man7.org/linux/man-pages/man2/recv.2.html>

LINUX-SEND — Linux/POSIX manual: send.2

<https://man7.org/linux/man-pages/man2/send.2.html>

LINUX-SOCKET — Linux/POSIX manual: socket.2

<https://man7.org/linux/man-pages/man2/socket.2.html>

OCI-DESCRIPTORS — Open Container Initiative: image descriptor specification

<https://github.com/opencontainers/image-spec/blob/main/descriptor.md>

OPENSSL-HOST — OpenSSL service identity verification

https://docs.openssl.org/3.5/man3/SSL_set1_host/

OPENSSL-X509 — OpenSSL 3.5: x509 command

<https://docs.openssl.org/3.5/man1/openssl-x509/>

OTEL-COLLECTOR — OpenTelemetry: Collector configuration

<https://opentelemetry.io/docs/collector/configuration/>

OTEL-CPP — OpenTelemetry: C++ instrumentation

<https://opentelemetry.io/docs/languages/cpp/instrumentation/>

PROM-ALERT — Prometheus: alerting rules

https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/

PROM-HISTOGRAMS — Prometheus: histograms and summaries

<https://prometheus.io/docs/practices/histograms/>

PROM-NAMING — Prometheus: metric and label naming

<https://prometheus.io/docs/practices/naming/>

SPDX — SPDX: software bill of materials specification

<https://spdx.github.io/spdx-spec/v2.3/>

SQLITE-BACKUP — SQLite online backup API

<https://www.sqlite.org/backup.html>

SQLITE-C — SQLite C/C++ interface specification

<https://www.sqlite.org/c3ref/intro.html>

SRE-ALERTING — Google SRE workbook: alerting on SLOs

<https://sre.google/workbook/alerting-on-slos/>

SRE-CANARY — Google SRE workbook: canarying releases

<https://sre.google/workbook/canarying-releases/>

SRE-SLO — Google SRE workbook: implementing SLOs

<https://sre.google/workbook/implementing-slos/>

TERRAFORM-DATA — Terraform: terraform_data resource

<https://developer.hashicorp.com/terraform/language/resources/terraform-data>

TERRAFORM-JSON — Terraform: JSON output format

<https://developer.hashicorp.com/terraform/internals/json-format>

TERRAFORM-VALIDATE — Terraform: input validation

<https://developer.hashicorp.com/terraform/language/validate>

TLS13 — IETF RFC8446: TLS1.3

<https://www.rfc-editor.org/rfc/rfc8446.html>

W3C-TRACE — W3C: Trace Context

<https://www.w3.org/TR/trace-context/>